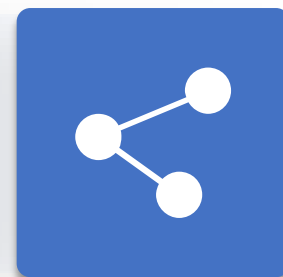




人工智能引论

第6章 神经网络

授课教师：李静瑶





录

1

什么是神经网络

2

神经网络的训练

3

卷积神经网络

4

梯度消失问题

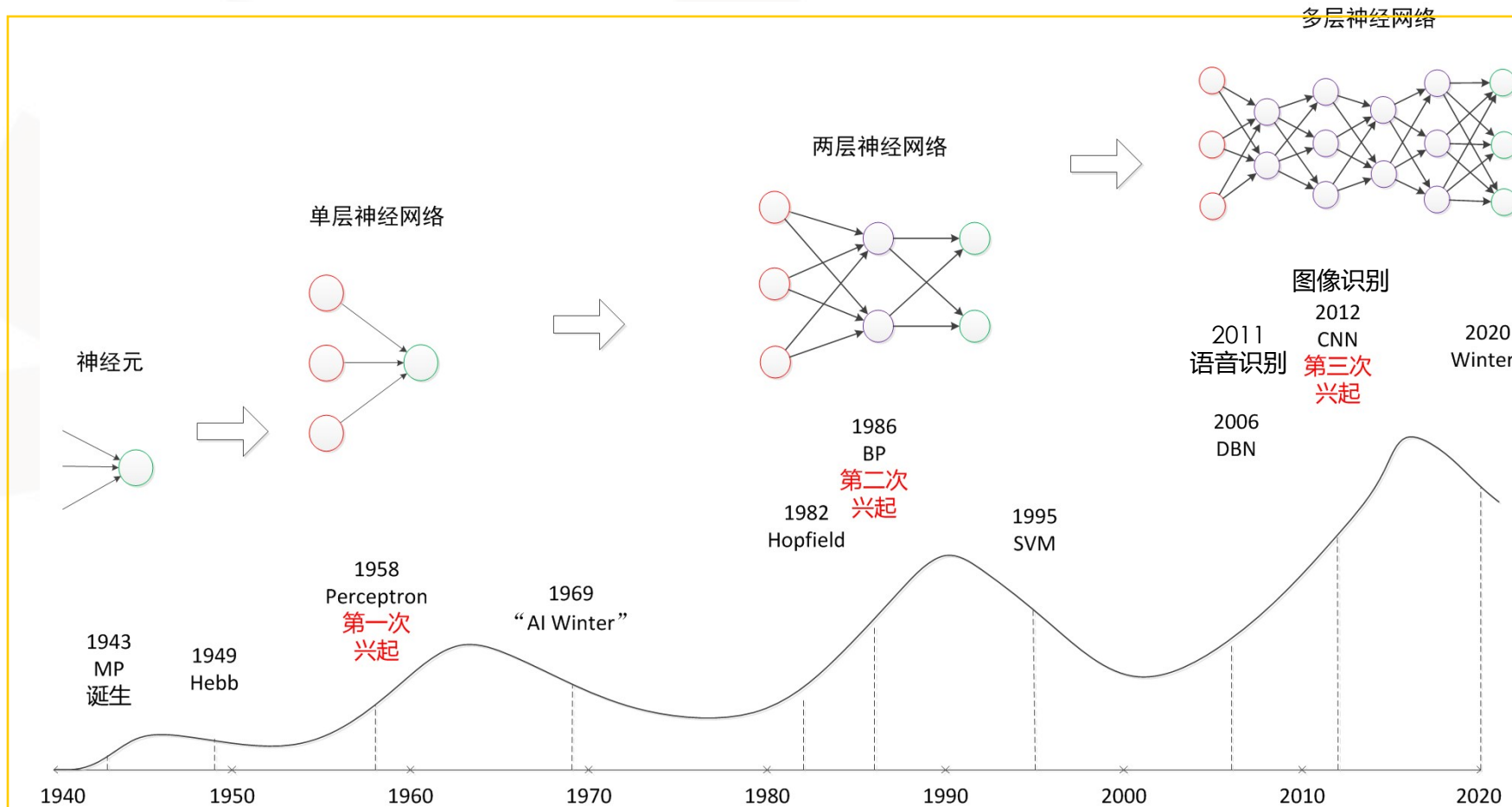
5

过拟合问题

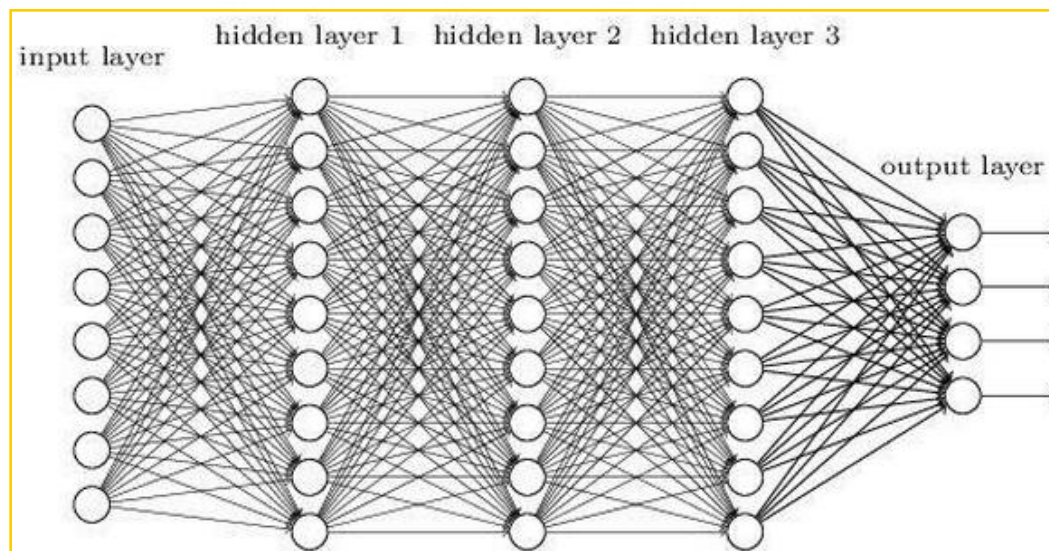
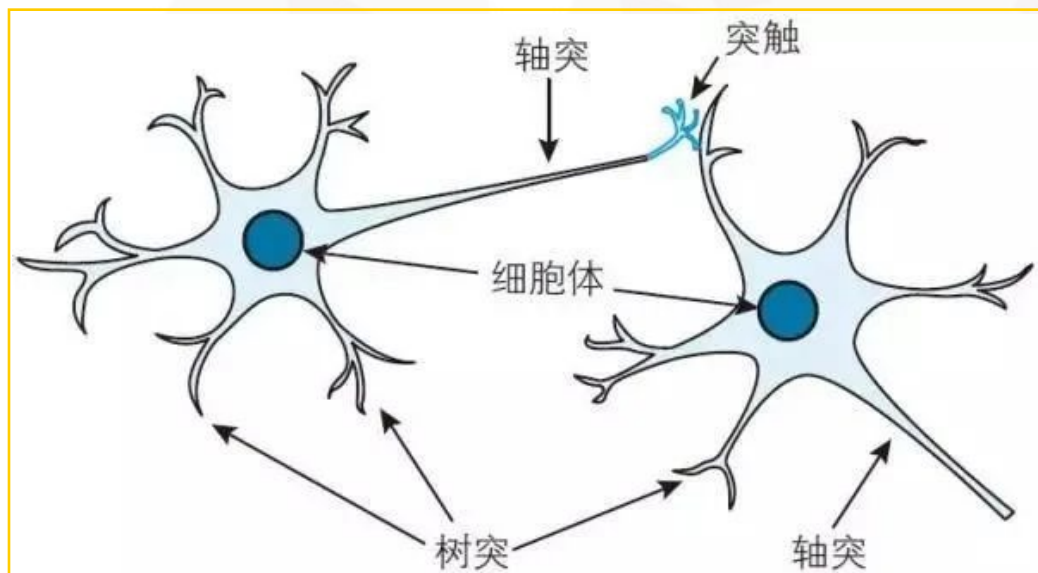
什么是神经网络



神经网络是如何实现的



人工神经网络 (ANN—Artificial Neural Network或者NN)



什么是神经网络



数字识别问题

1	1	1	1	1	1	1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	0	0	0	0	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	0	0	0	0	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	0	0

数字3的图像

1	1	1	1	1	1	1	1	1	1	1	1	1	-1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	-1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	-1	-1

数字3的模式

什么是神经网络



数字识别问题

0	0	1	1	1	1	1	1	1	1	1	1	1	0	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	0	0	0	0	0	0	0	0	0	1	1	1
1	1	1	0	0	0	0	0	0	0	0	0	1	1	1
1	1	1	0	0	0	0	0	0	0	0	0	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	1	1	1	1	1	1	1	1	1	1	1	0	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	0	0	0	0	0	0	0	0	0	1	1	1
1	1	1	0	0	0	0	0	0	0	0	0	1	1	1
1	1	1	0	0	0	0	0	0	0	0	0	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	1	1	1	1	1	1	1	1	1	1	1	0	0

数字8的图像

1	1	1	1	1	1	1	1	1	1	1	1	1	-1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	-1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	-1	-1

数字3的模式

什么是神经网络

模式匹配 (按位乘 / 点积)

0	1	1	1	0
1	0	0	0	1
0	1	1	1	0
1	0	0	0	1
0	1	1	1	0

•

1	1	1	1	-1
-1	-1	-1	-1	1
1	1	1	1	-1
-1	-1	-1	-1	1
1	1	1	1	-1

=

$$\begin{aligned}
 &0 \times 1 + 1 \times 1 + 1 \times 1 + 1 \times 1 + 0 \times (-1) + \\
 &1 \times (-1) + 0 \times (-1) + 0 \times (-1) + 0 \times (-1) + 1 \times 1 + \\
 &0 \times 1 + 1 \times 1 + 1 \times 1 + 1 \times 1 + 0 \times (-1) + \\
 &1 \times (-1) + 0 \times (-1) + 0 \times (-1) + 0 \times (-1) + 1 \times 1 + \\
 &0 \times 1 + 1 \times 1 + 1 \times 1 + 1 \times 1 + 0 \times (-1) \\
 &= 9
 \end{aligned}$$

$$net = w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + w_n \cdot x_n$$

其中: $w_i (i = 1, 2, \dots, n)$: 表示数字模式

$x_i (i = 1, 2, \dots, n)$: 表示数字图像

什么是神经网络



数字3、数字8与模式3的匹配

1	1	1	1	1	1	1	1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	0

•

1	1	1	1	1	1	1	1	1	1	1	1	1	-1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	-1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	-1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1

= 143

0	0	1	1	1	1	1	1	1	1	1	1	1	0	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	0	0	0	0	0	0	0	0	0	1	1	1
1	1	1	0	0	0	0	0	0	0	0	0	1	1	1
1	1	1	0	0	0	0	0	0	0	0	0	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	1	1	1	1	1	1	1	1	1	1	1	0	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	0	0	0	0	0	0	0	0	0	1	1	1
1	1	1	0	0	0	0	0	0	0	0	0	1	1	1
1	1	1	0	0	0	0	0	0	0	0	0	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	1	1	1	1	1	1	1	1	1	1	1	0	0

•

1	1	1	1	1	1	1	1	1	1	1	1	1	-1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	-1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1

= 115

什么是神经网络



存在的问题

- 笔画多少带来的问题
 - 1的笔画少
 - 8的笔画多
- 如何评判匹配的程度?

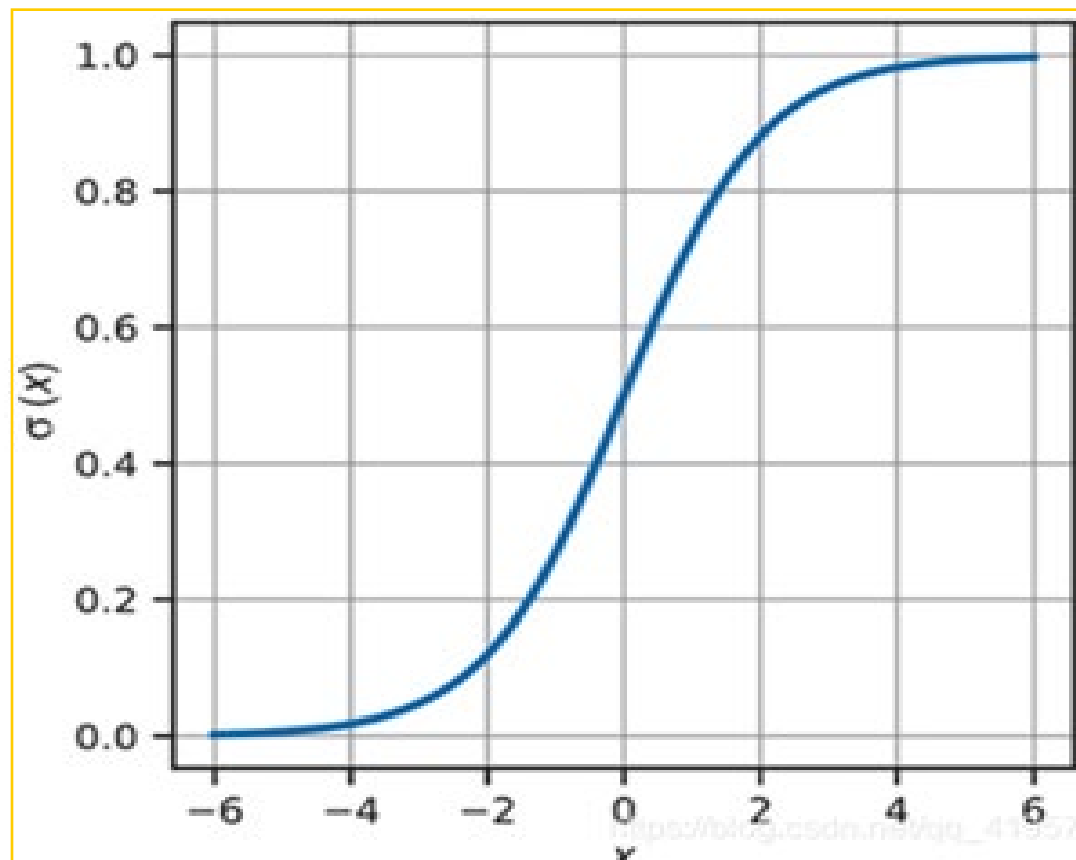
什么是神经网络



使用Sigmoid函数

- 使用Sigmoid函数将匹配结果变换到0-1之间

$$\sigma(net) = \frac{1}{1+e^{-net}}$$

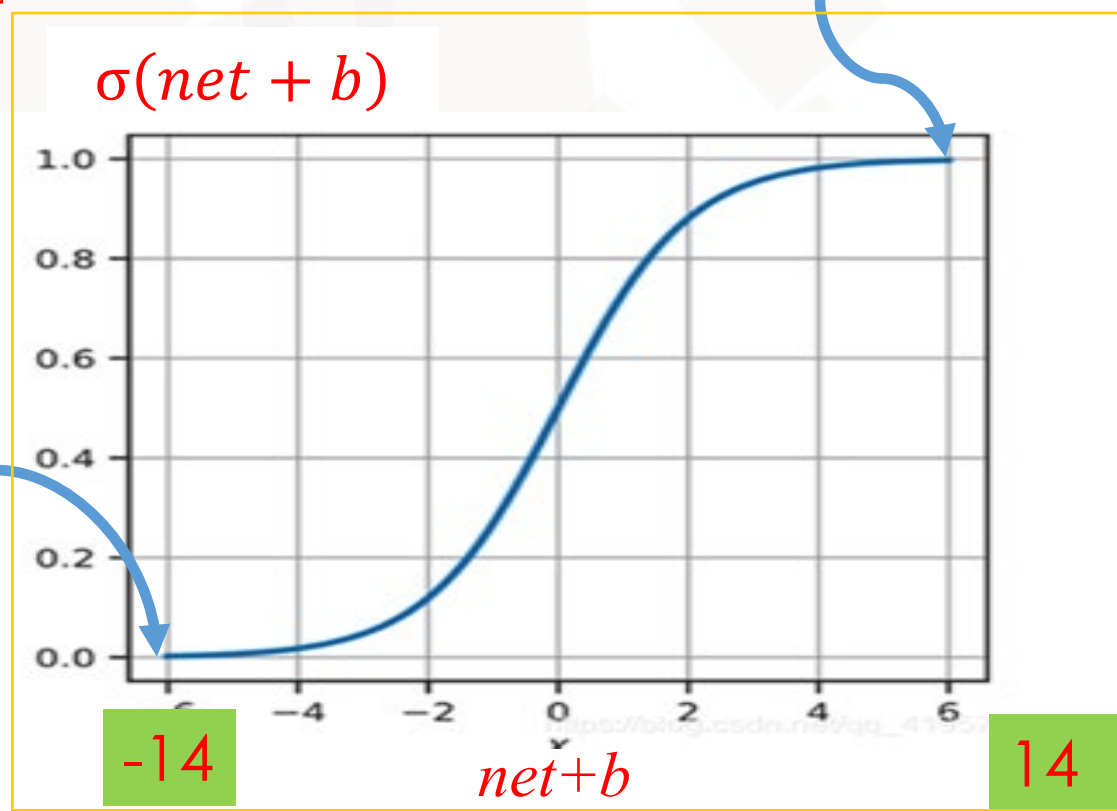
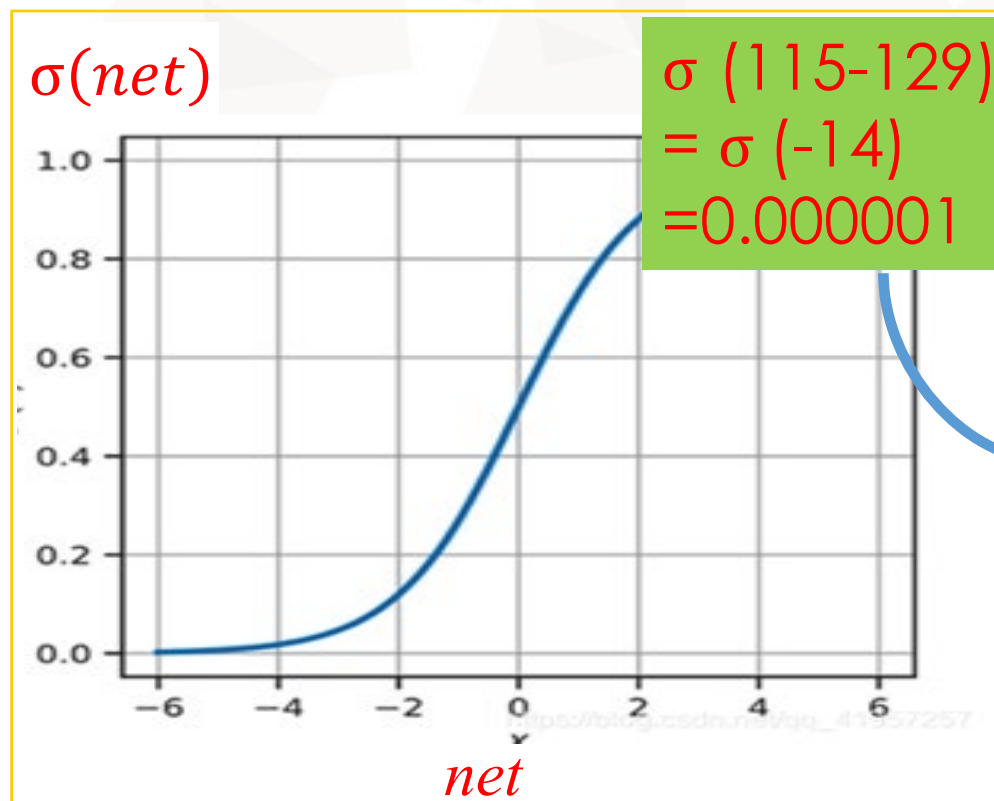


什么是神经网络



增加偏置项

- 通过增加偏置项让Sigmoid函数平移



什么是神经网络



增加偏置项

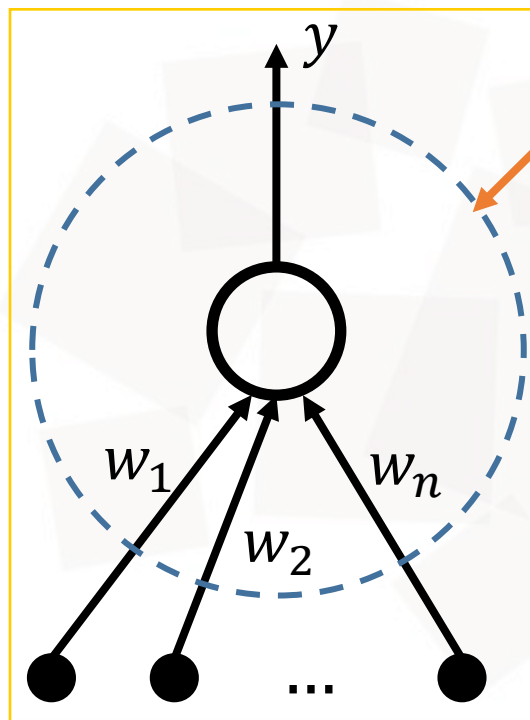
- 模式匹配:

$$net = w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + w_n \cdot x_n + b$$

$$y = \text{sigmoid}(net)$$

什么是神经网络

图示表示



神经元

模式

$$net = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$$

$$y = \text{sigmoid}(net)$$

x_1

x_2

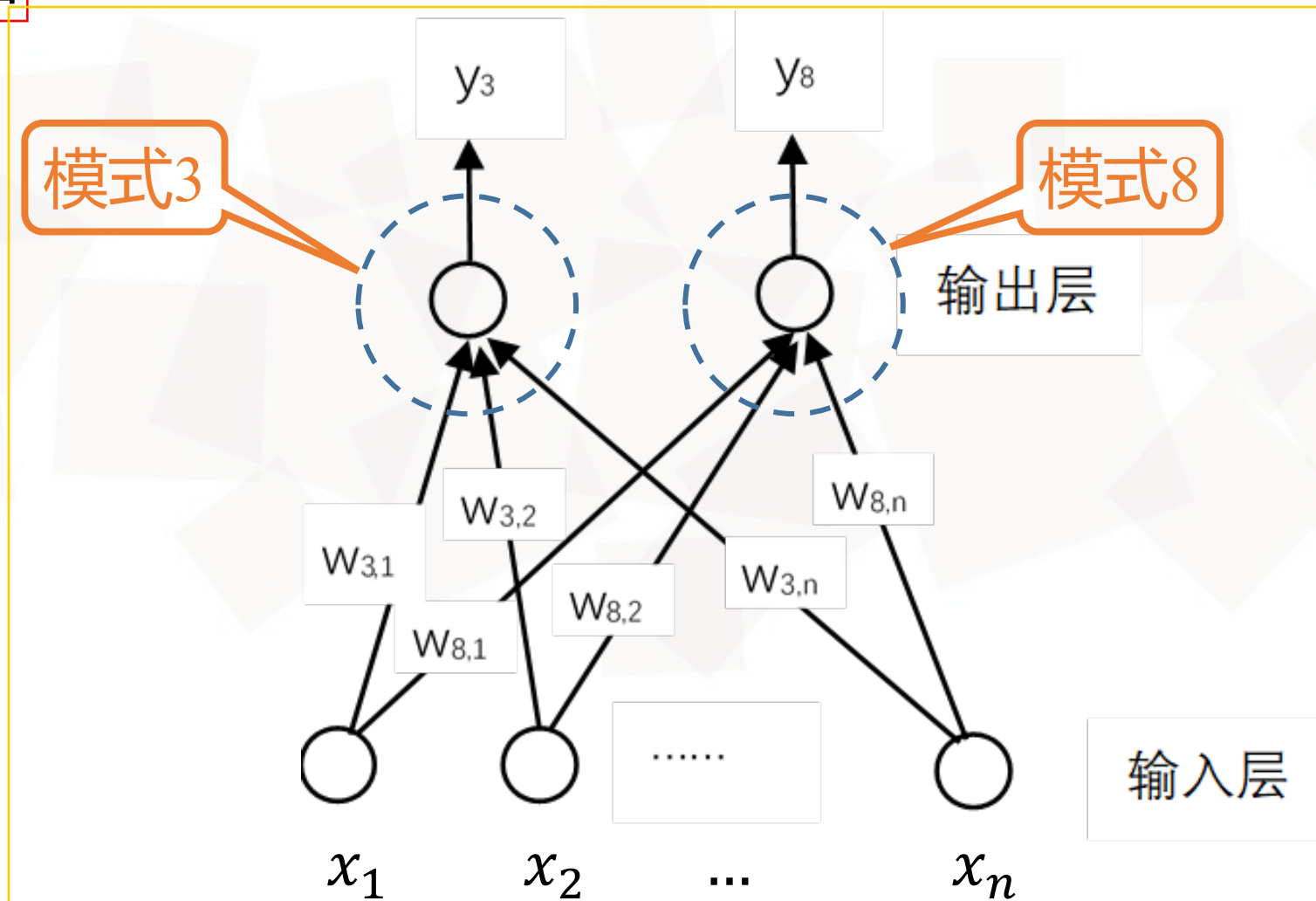
...

x_n

什么是神经网络



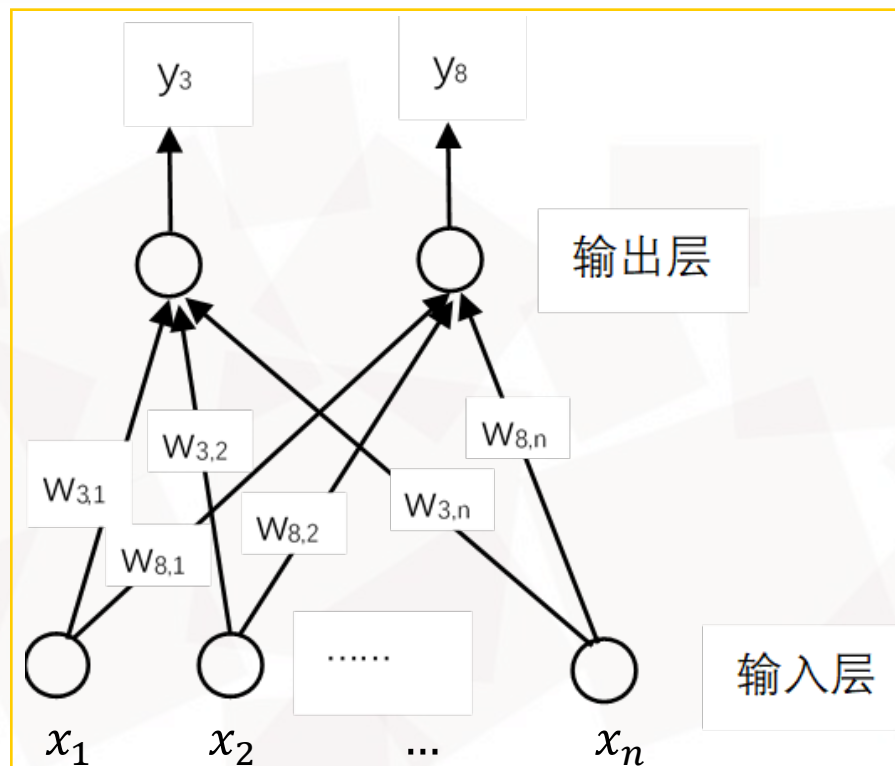
神经网络



什么是神经网络



神经网络



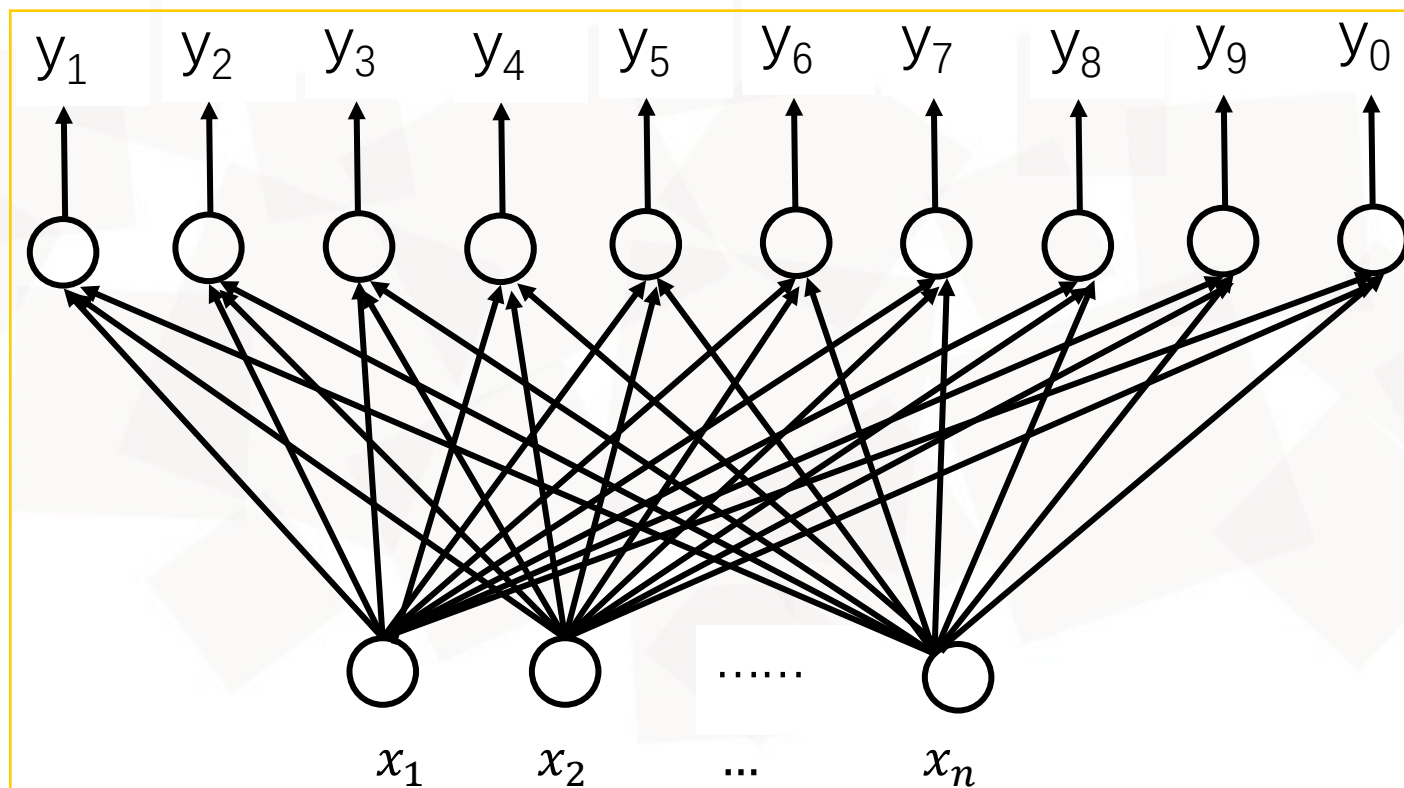
$$y_3 = \text{sigmoid}(w_{3,1} \cdot x_1 + w_{3,2} \cdot x_2 + \dots + w_{3,n} \cdot x_n + b_3)$$

$$y_8 = \text{sigmoid}(w_{8,1} \cdot x_1 + w_{8,2} \cdot x_2 + \dots + w_{8,n} \cdot x_n + b_8)$$

什么是神经网络

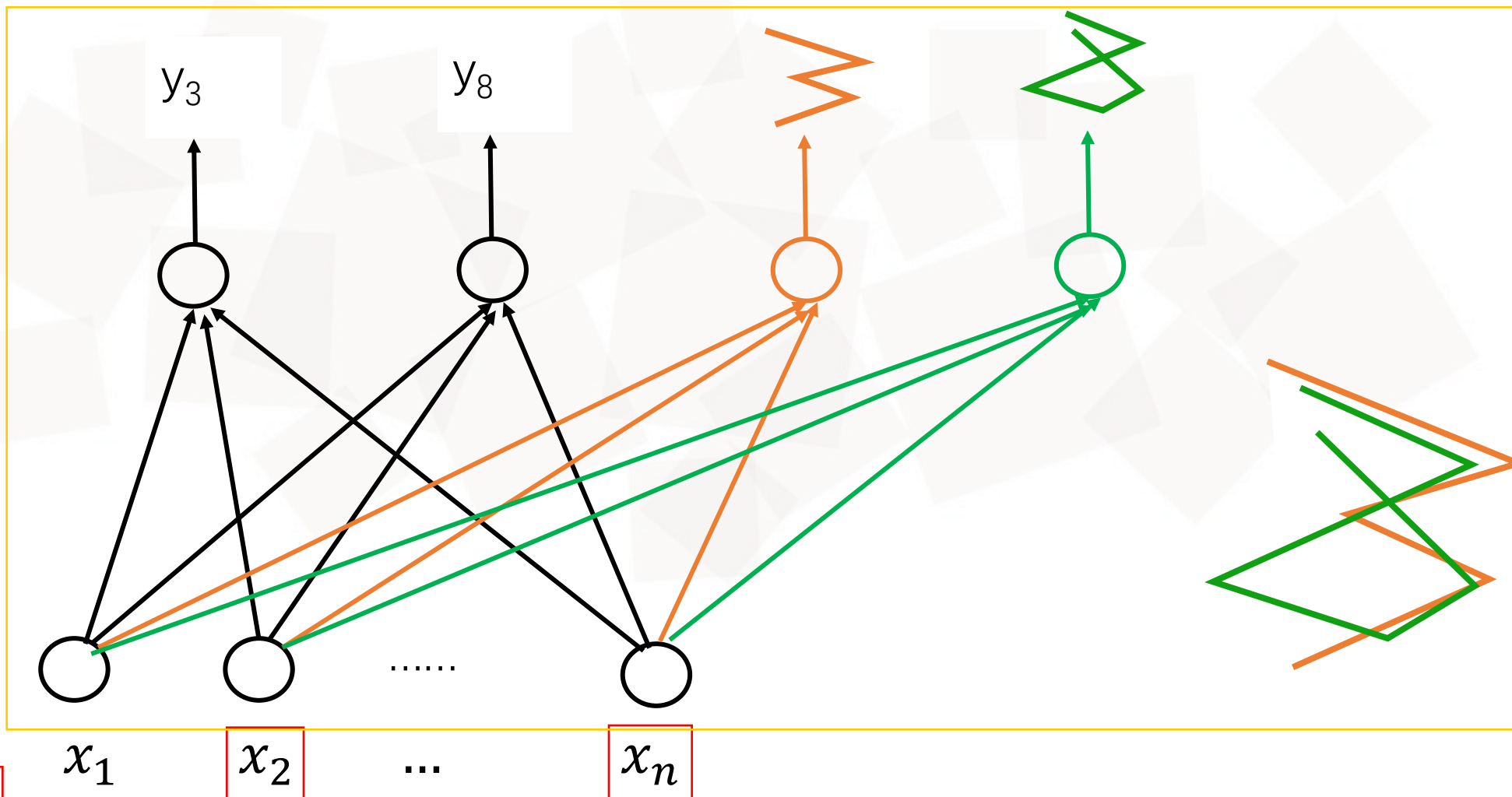


数字识别神经网络



$$y_i = \text{sigmoid}(w_{i,1} \cdot x_1 + w_{i,2} \cdot x_2 + \dots + w_{i,n} \cdot x_n + b_i)$$

神经网络的横向扩展——增加模式



什么是神经网络



神经网络的纵向扩展

局部模式

不相同

相同

1	1	1	1	1	1	1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0	0	0	0	1	1	1
0	0	0	0	0	0	0	0	0	0	0	1	1	1
0	0	0	0	0	0	0	0	0	0	0	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	0	0	0	0	0	1	1	1
0	0	0	0	0	0	0	0	0	0	0	1	1	1
0	0	0	0	0	0	0	0	0	0	0	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	0	0

0	0	1	1	1	1	1	1	1	1	1	1	0	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	0	0	0	0	0	0	0	0	1	1	1
1	1	1	0	0	0	0	0	0	0	0	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	1	1	1	1	1	1	1	1	1	1	0	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	0	0	0	0	0	0	0	0	1	1	1
1	1	1	0	0	0	0	0	0	0	0	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	1	1	1	1	1	1	1	1	1	0	0	0

什么是神经网络



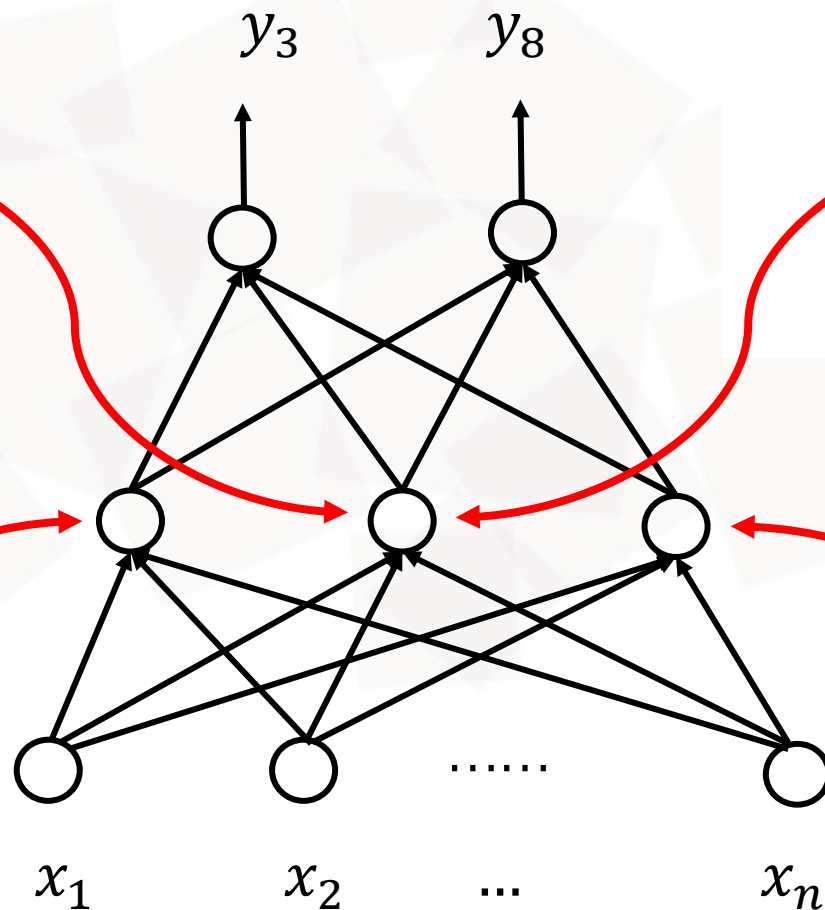
让神经网络更深——模式组合

3
的
右
部

1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
1	1	1	1	1	1	1	0
1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	0
1	1	1	1	1	1	0	0

3
的
左
部

1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1



8
的
右
部

1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1
0	0	0	0	0	1	1	1
0	0	0	0	0	1	1	1
0	0	0	0	0	1	1	1
1	1	1	1	1	1	1	0
1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	0
0	0	0	0	0	1	1	1
0	0	0	0	0	1	1	1
0	0	0	0	0	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	0
1	1	1	1	1	1	0	0

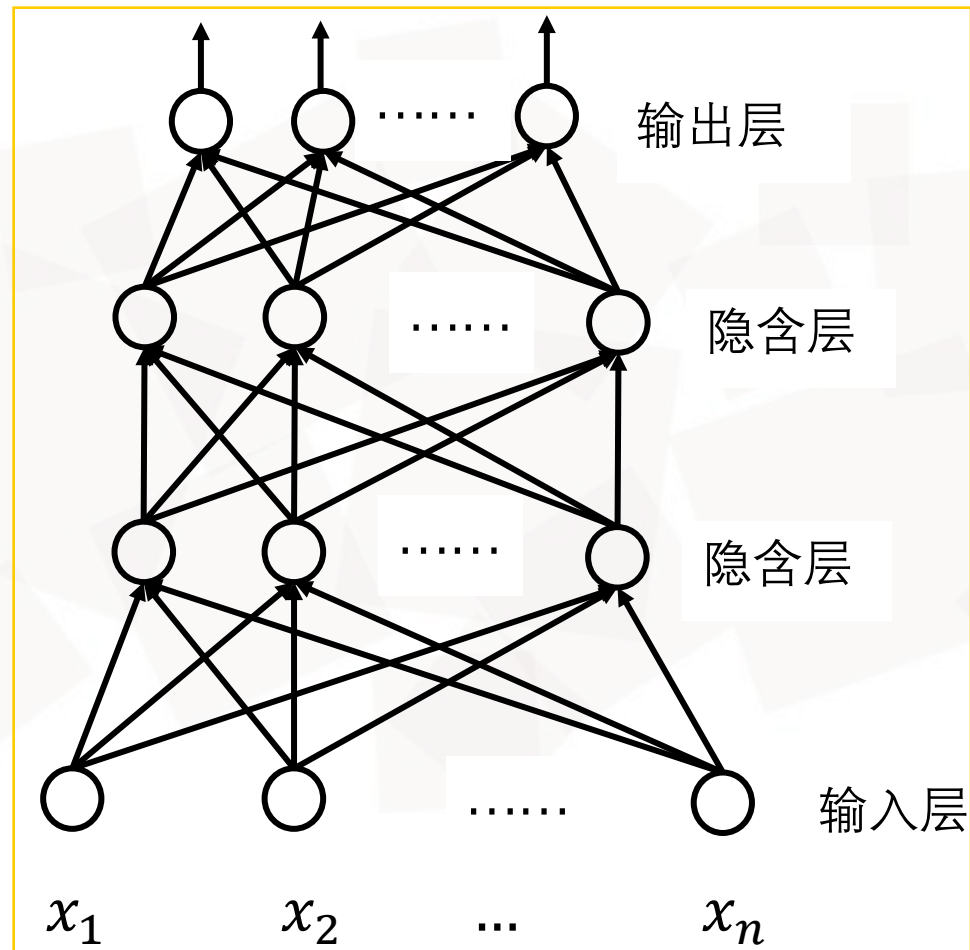
8
的
左
部

0	0	1	1	1	1	1	1
0	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	0	0	0	0	0
1	1	1	0	0	0	0	0
1	1	1	0	0	0	0	0
0	1	1	1	1	1	1	1
0	0	1	1	1	1	1	1
0	1	1	1	1	1	1	1
1	1	1	0	0	0	0	0
1	1	1	0	0	0	0	0
1	1	1	0	0	0	0	0
1	1	1	1	1	1	1	1
0	1	1	1	1	1	1	1
0	0	1	1	1	1	1	1

什么是神经网络



多层神经网络



什么是神经网络



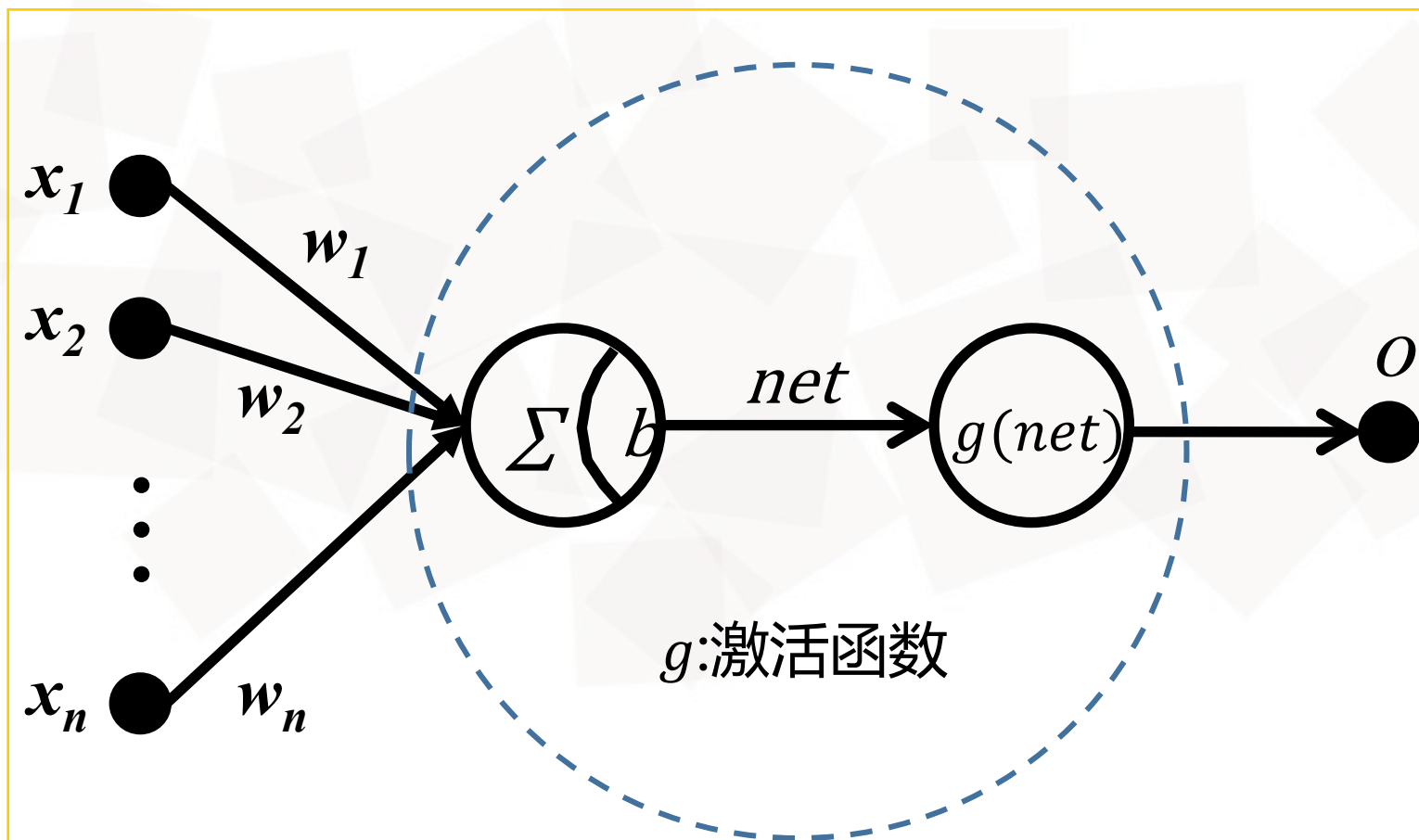
如何获得模式?

- 模式通过神经元的连接权重表示
- 通过训练样本，自动学习权重，也就是模式
- 不是人工设计!
- 学习到的模式是一种隐含表达，并不像我们举例的这样清晰

什么是神经网络



神经元与神经网络

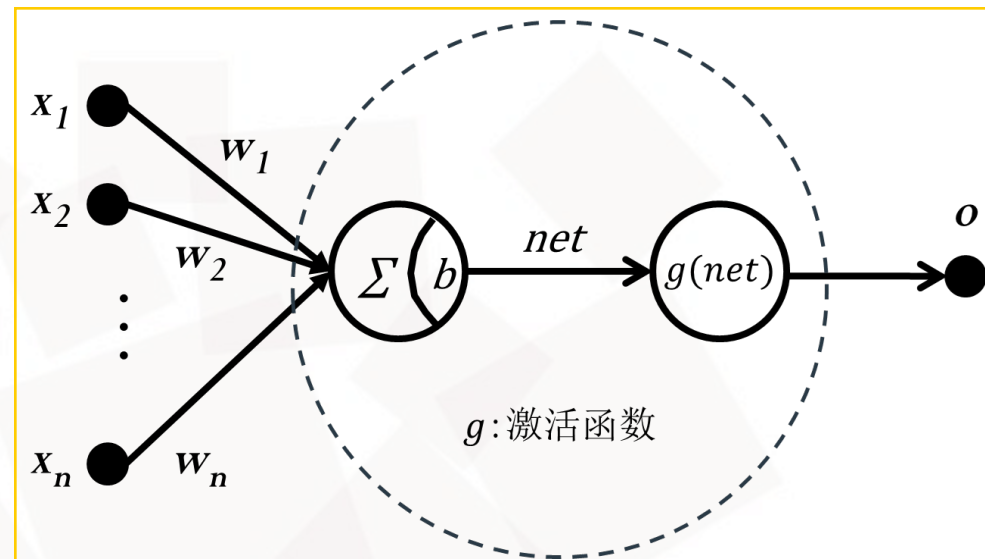


什么是神经网络

神经元

$$\begin{aligned} net &= w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + w_n \cdot x_n + b \\ &= \sum_{i=1}^n w_i \cdot x_i + b \end{aligned}$$

$$o = g(net)$$



令: $x_0 = 1, w_0 = b$

则:
$$\begin{aligned} net &= w_0 \cdot x_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + w_n \cdot x_n \\ &= \sum_{i=0}^n w_i \cdot x_i \end{aligned}$$

什么是神经网络



神经元的向量表示

- 输入: $x = [x_0, x_1, \dots, x_n]$
- 权重: $w = [w_0, w_1, \dots, w_n]$

$$net = w \cdot x$$

$$o = g(net)$$

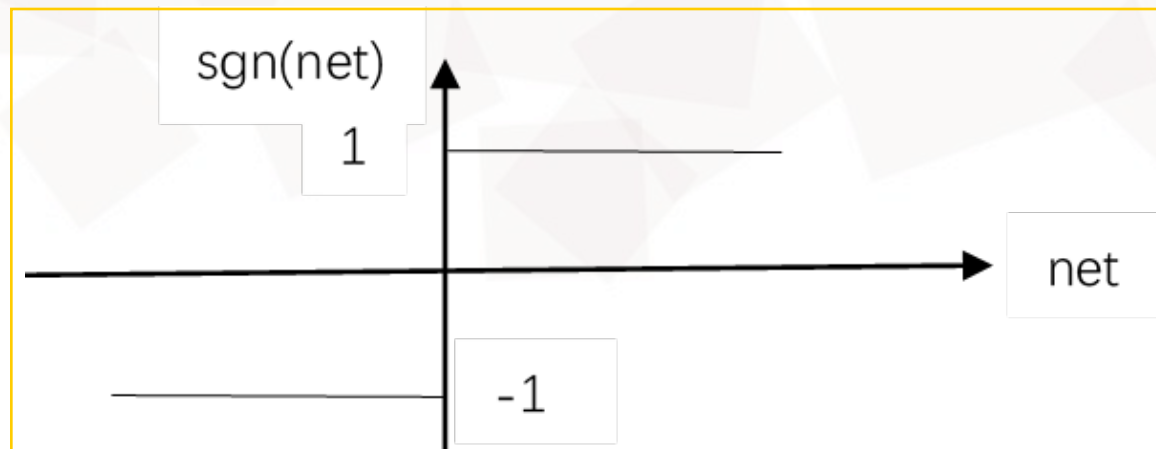
什么是神经网络



激活函数

• 符号函数

$$g(net) = \text{sgn}(net) = \begin{cases} 1, & \text{当 } net \geq 0 \\ -1, & \text{当 } net < 0 \end{cases}$$



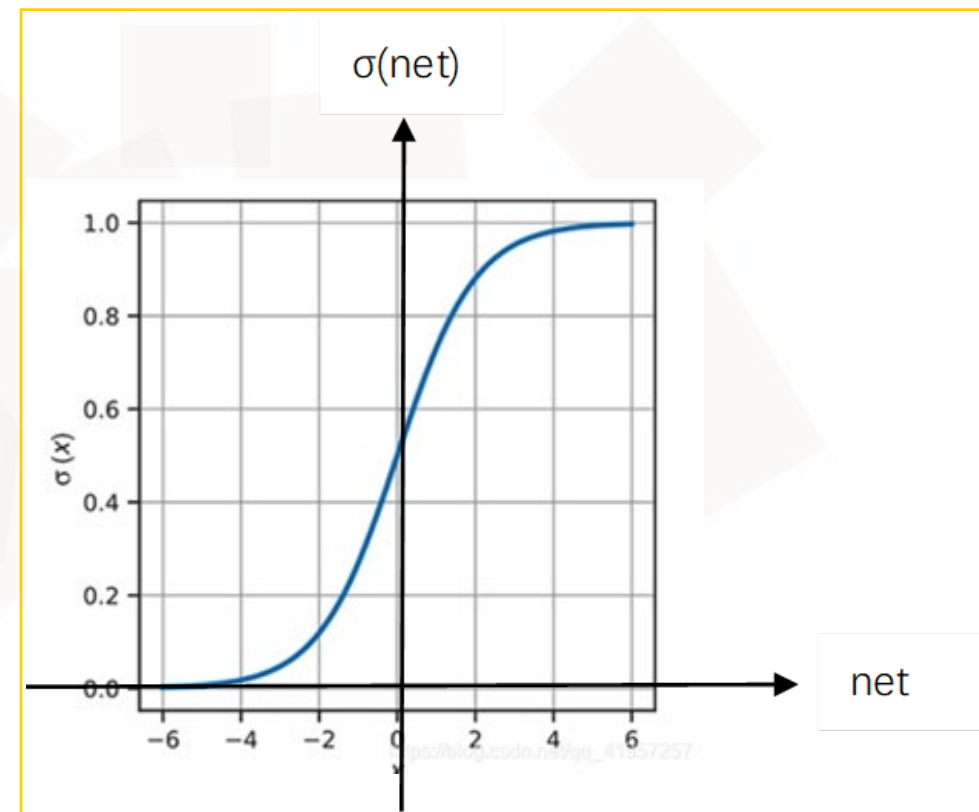
什么是神经网络



激活函数

- Sigmoid函数

$$g(net) = \sigma(net) = \frac{1}{1 + e^{-net}}$$



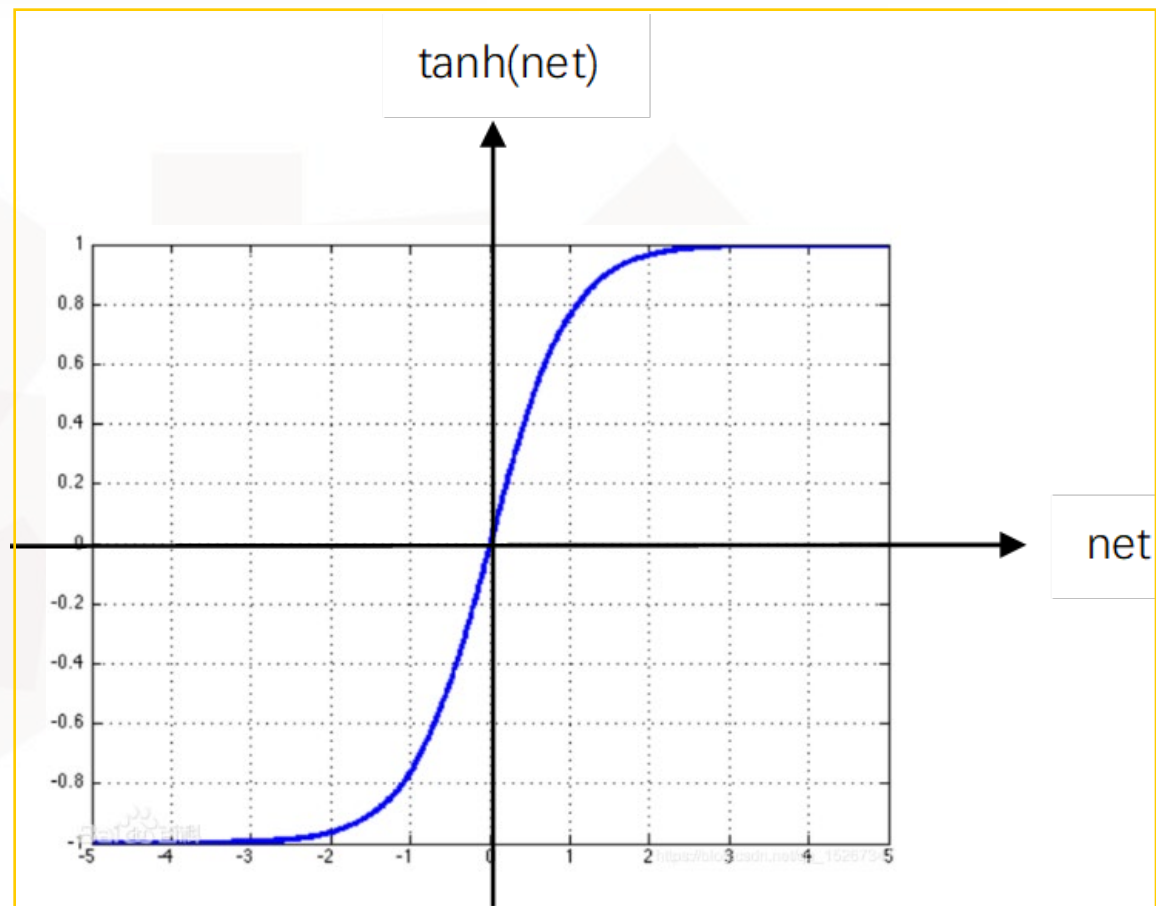
什么是神经网络



激活函数

- 双曲正切函数

$$g(net) = \tanh(net) = \frac{e^{net} - e^{-net}}{e^{net} + e^{-net}}$$



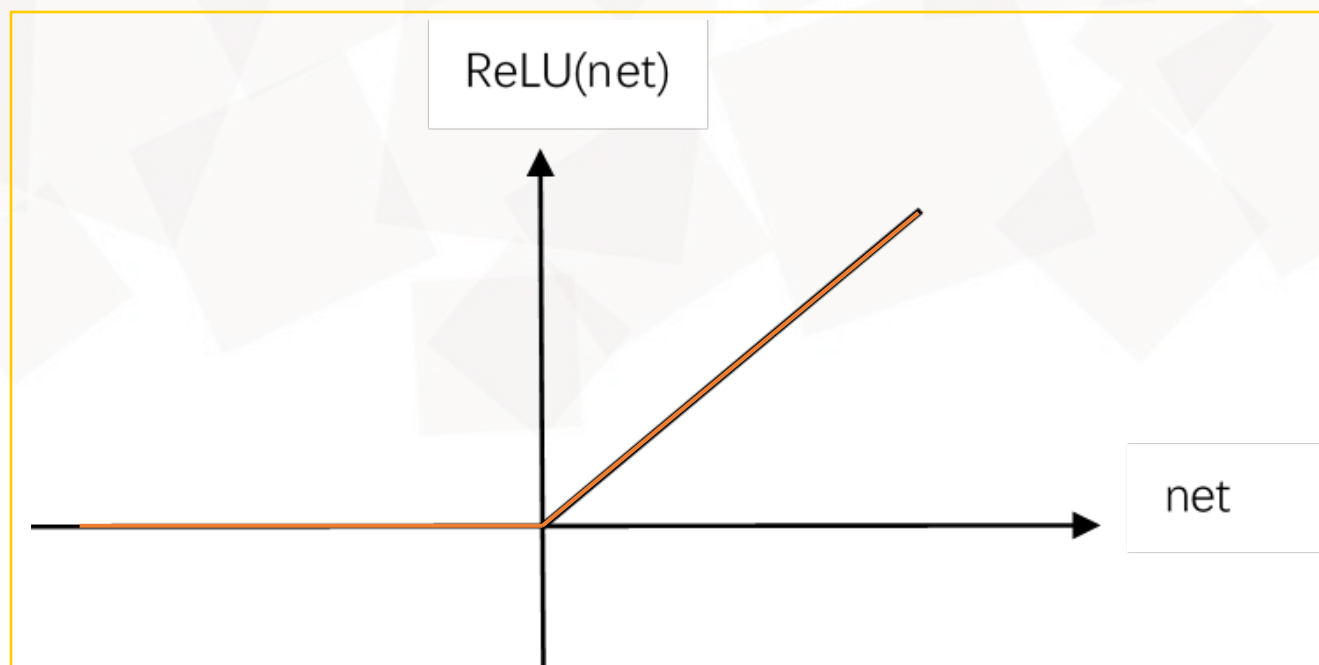
什么是神经网络



激活函数

- 线性整流函数

$$g(net) = \text{ReLU}(net) = \max(0, net)$$



什么是神经网络



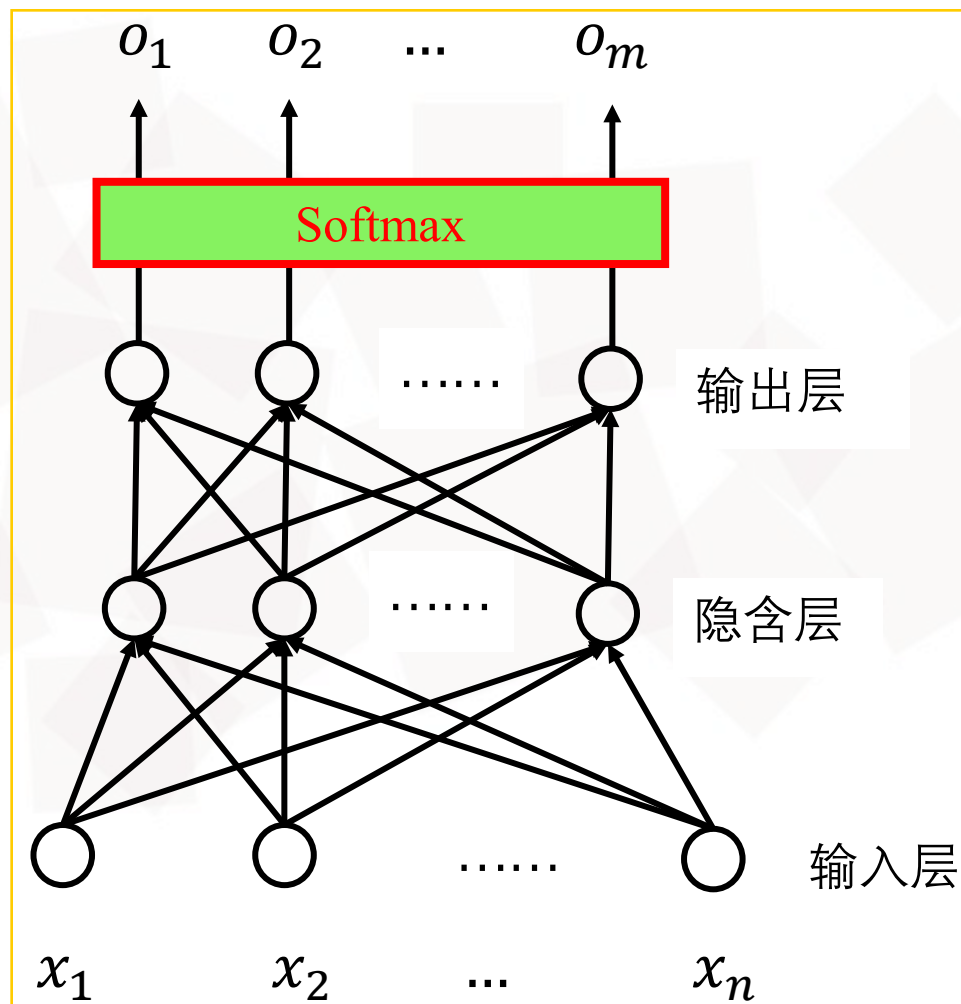
激活函数

- Softmax函数

$$o_k = \frac{e^{net_k}}{\sum_{i=1}^m e^{net_i}}$$

$$0 \leq o_k \leq 1$$

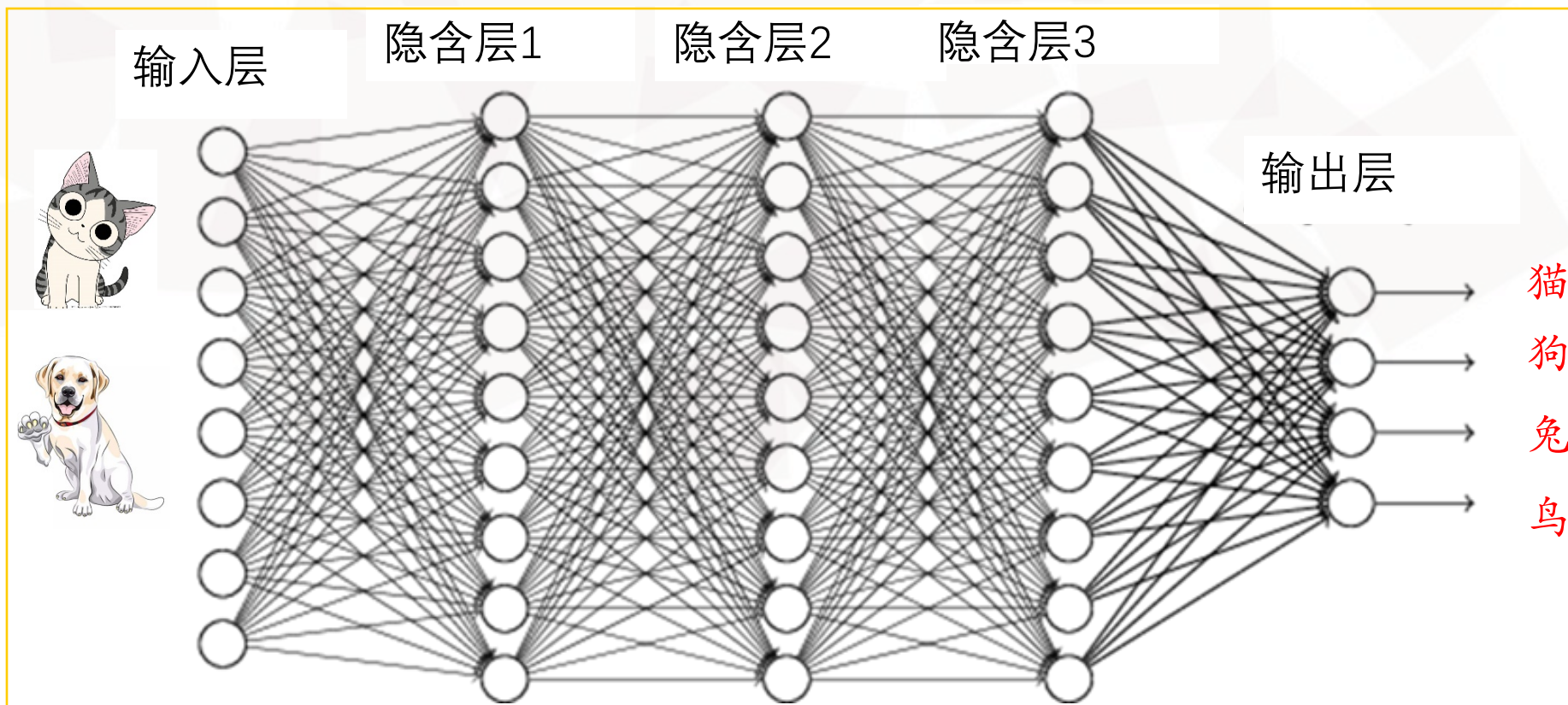
$$\sum_{k=1}^m o_k = 1$$



什么是神经网络

全连接网络

- 术语：全连接网络，前馈网络，多层感知机，全连接层，稠密层



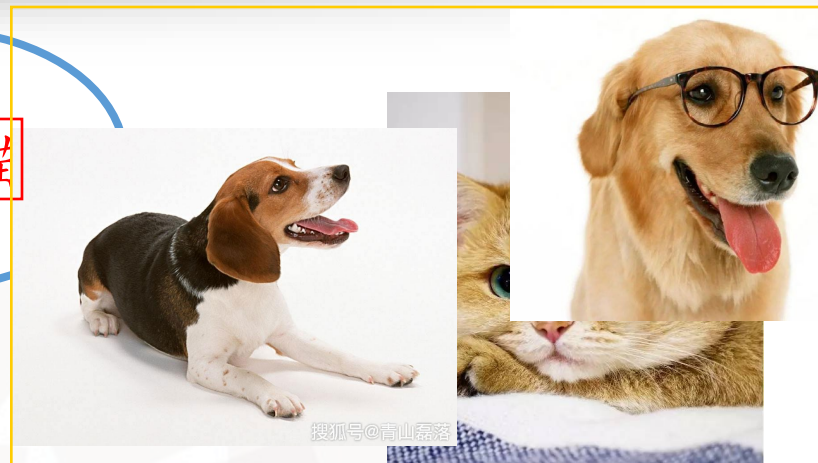
如何认识小动物



建立数据集

- 收集各种动物的照片
 - 不同姿势
 - 不同角度
 - 不同大小
- 数据标注
 - 每张照片标注上动物的名称
- 训练集与测试集
 - 样本

这个是猫



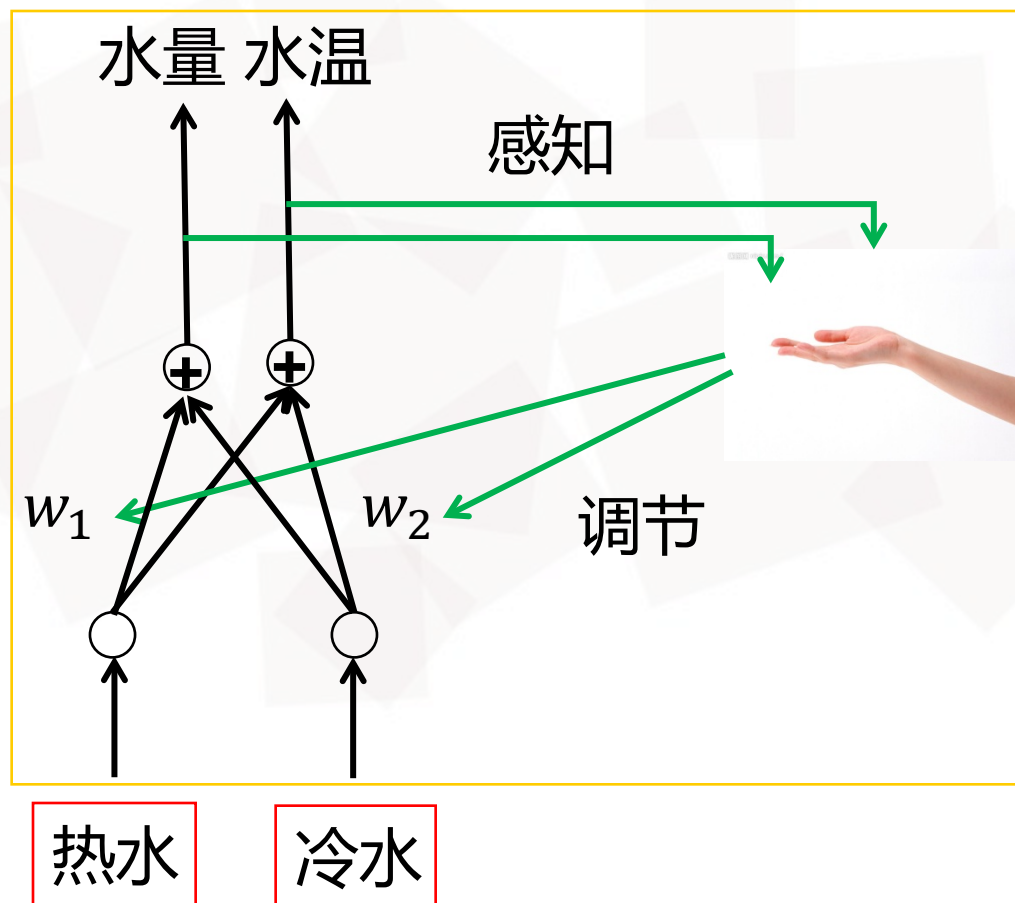
这个是狗!



如何训练



淋浴器示意图



如何评价调节效果——损失函数

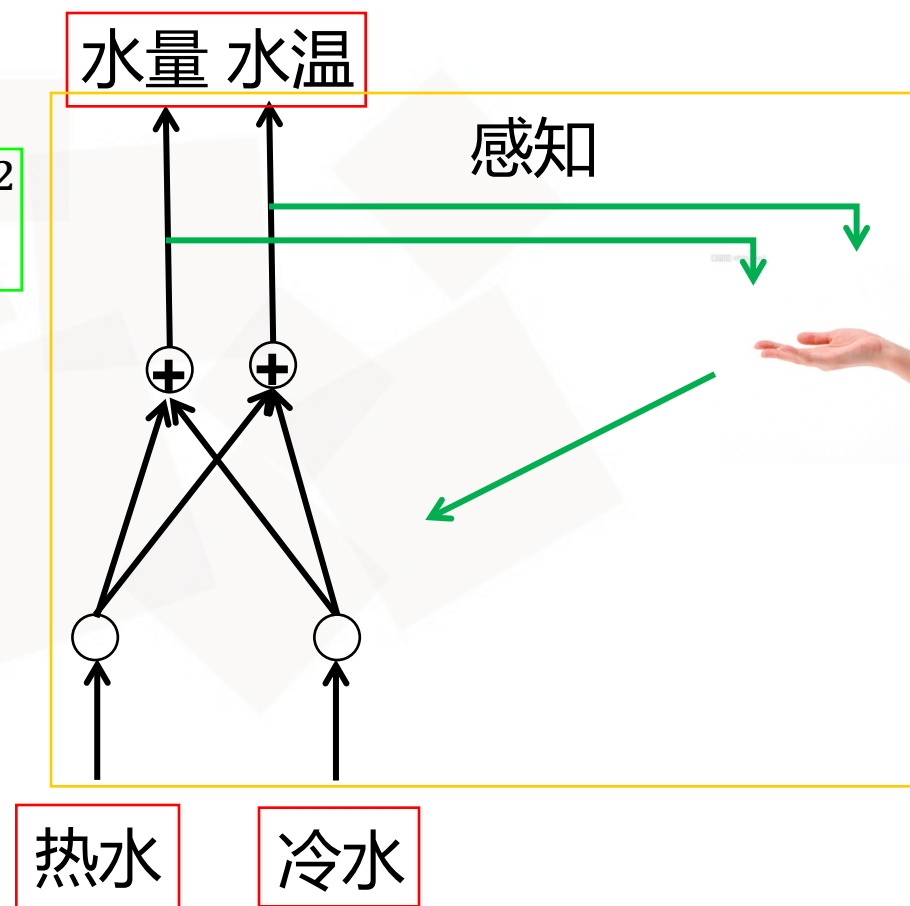
$$E(\text{阀门}) = (t_{\text{水温}} - o_{\text{水温}})^2 + (t_{\text{水量}} - o_{\text{水量}})^2$$

$t_{\text{水温}}$ ：希望水温

$t_{\text{水量}}$ ：希望水量

$o_{\text{水温}}$ ：实际水温

$o_{\text{水量}}$ ：实际水量



损失函数 — 误差平方和

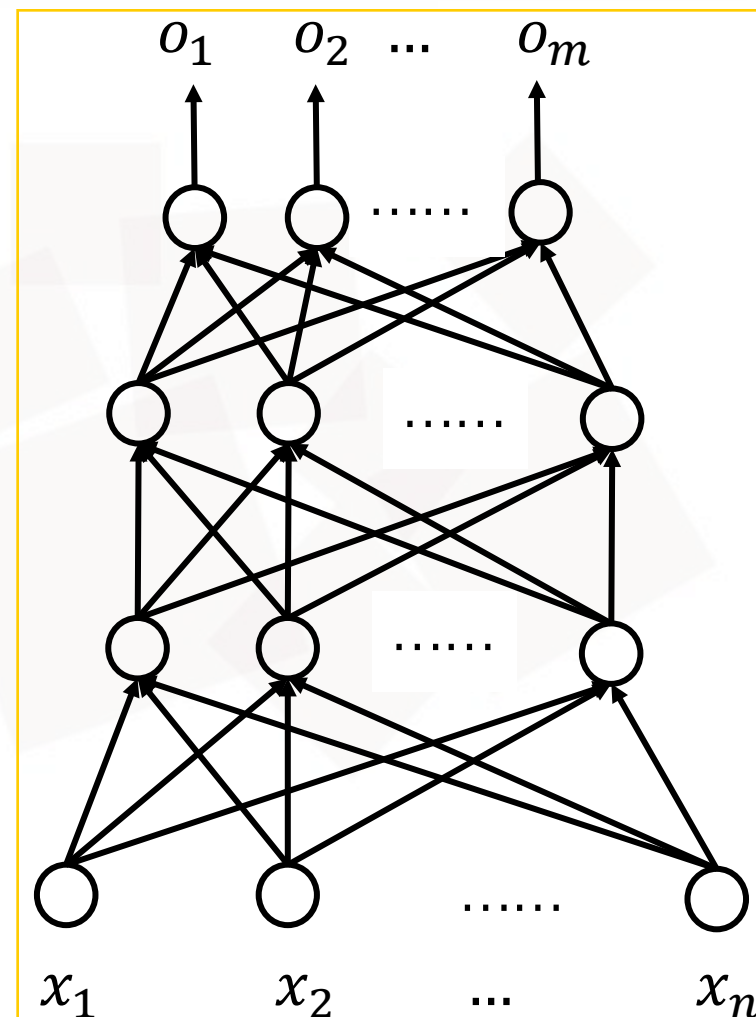
- 对样本d的误差

$$E_d(w) = \frac{1}{2} \sum_{k=1}^m (t_{kd} - o_{kd})^2$$

- 对所有样本的误差

$$\begin{aligned} E(w) &= \frac{1}{2} \sum_{d=1}^n E_d(w) \\ &= \frac{1}{2} \sum_{d=1}^n \sum_{k=1}^m (t_{kd} - o_{kd})^2 \end{aligned}$$

- t_{kd} : 对应样本d的第k个希望输出
- o_{kd} : 对应样本d的第k个实际输出

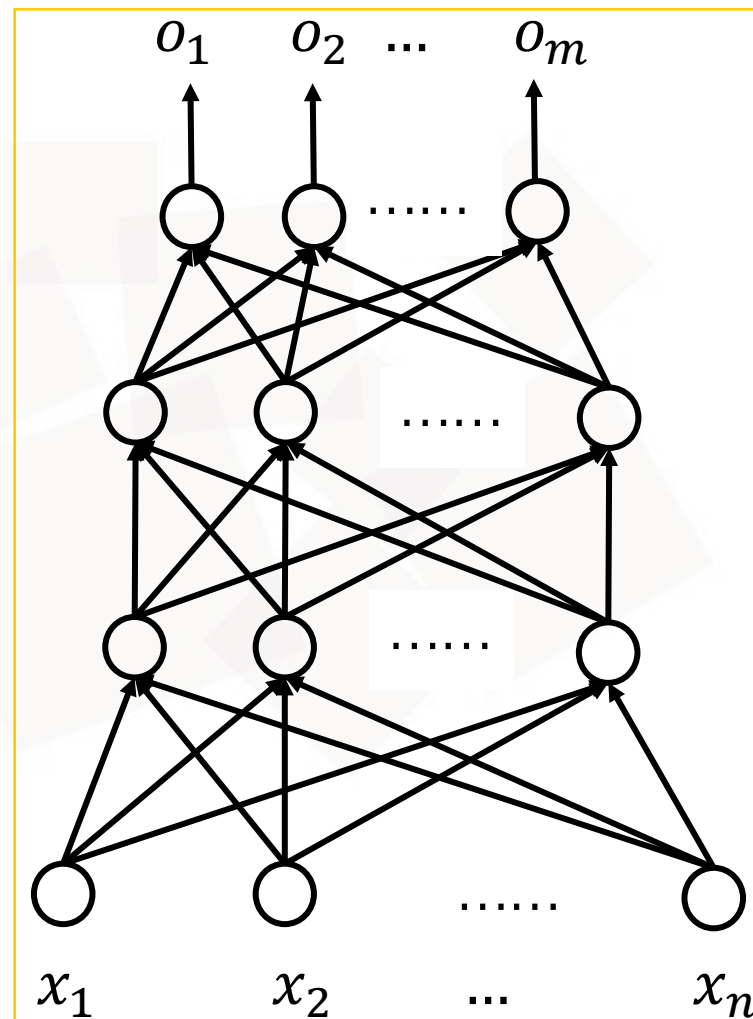


如何训练

- 损失函数最小化问题

$$E_d(w) = \frac{1}{2} \sum_{k=1}^m (t_{kd} - o_{kd})^2$$

$$\begin{aligned} E(w) &= \frac{1}{2} \sum_{d=1}^n E_d(w) \\ &= \frac{1}{2} \sum_{d=1}^n \sum_{k=1}^m (t_{kd} - o_{kd})^2 \end{aligned}$$



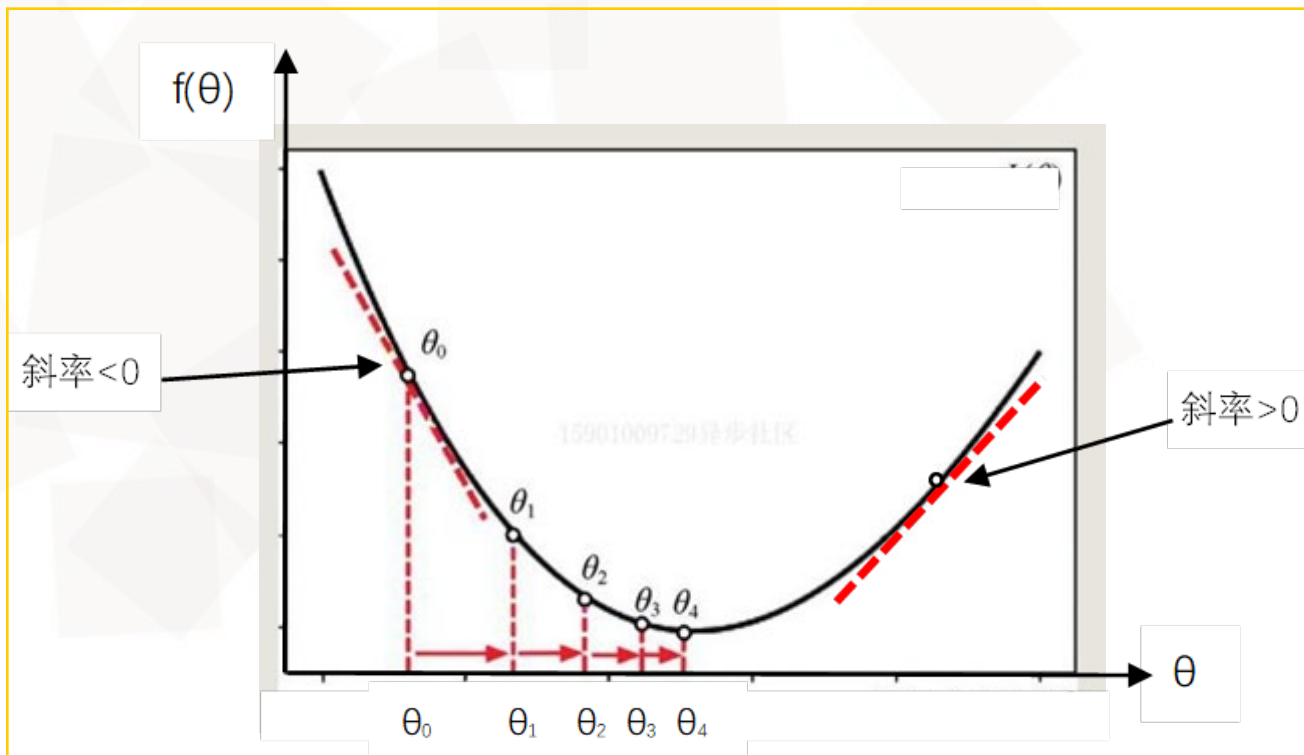
梯度下降法

- 求 $f(\theta)$ 的最小值
- 迭代逼近

$$\theta_{i+1} = \theta_i + \Delta \theta_i$$

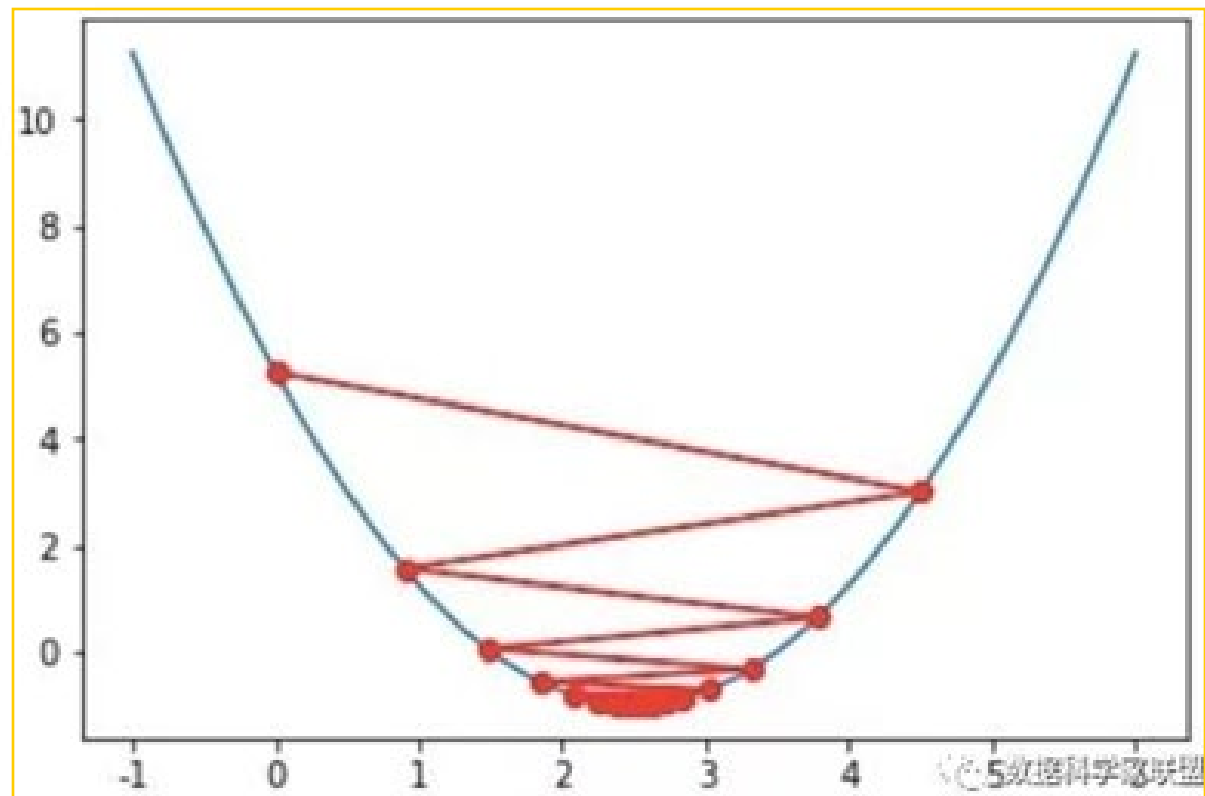
- 两个问题:
 - 修改量的大小
 - 修改的方向

$$\Delta \theta_i = - \frac{df}{d\theta}$$



梯度下降法

- 震荡问题
- 步长 η
- $\Delta \theta_i = -\eta \frac{df}{d\theta}$



梯度下降法

- 迭代公式:

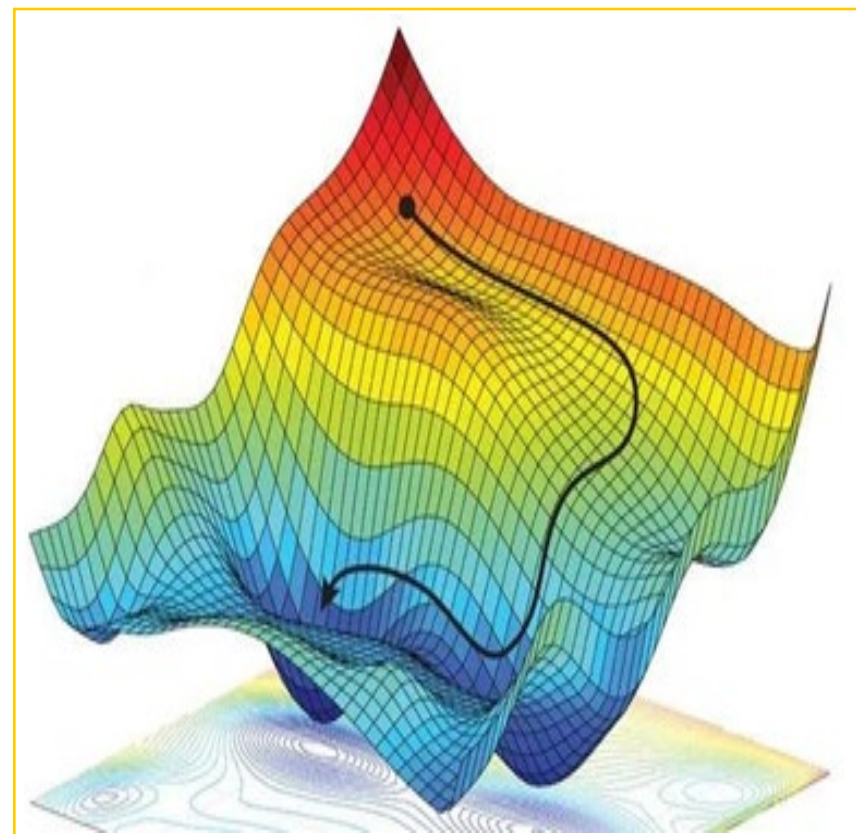
$$w_i^{new} = w_i^{old} + \Delta w_i$$
$$\Delta w_i = -\eta \frac{\partial E(w)}{\partial w_i}$$

- 梯度:

$$\nabla_w E(w) = \left[\frac{\partial E(w)}{\partial w_1}, \frac{\partial E(w)}{\partial w_2}, \dots, \frac{\partial E(w)}{\partial w_n} \right]$$

$$w^{new} = w^{old} + \Delta w$$

$$\Delta w = -\eta \nabla_w E(w)$$





梯度下降算法

- 批量梯度下降算法
 - 每次处理全部样本
- 随机梯度下降算法
 - 每次处理一个样本
- 小批量梯度下降算法
 - 每次处理小批量的样本

$$E(w) = \frac{1}{2} \sum_{d=1}^n \sum_{k=1}^m (t_{kd} - o_{kd})^2$$

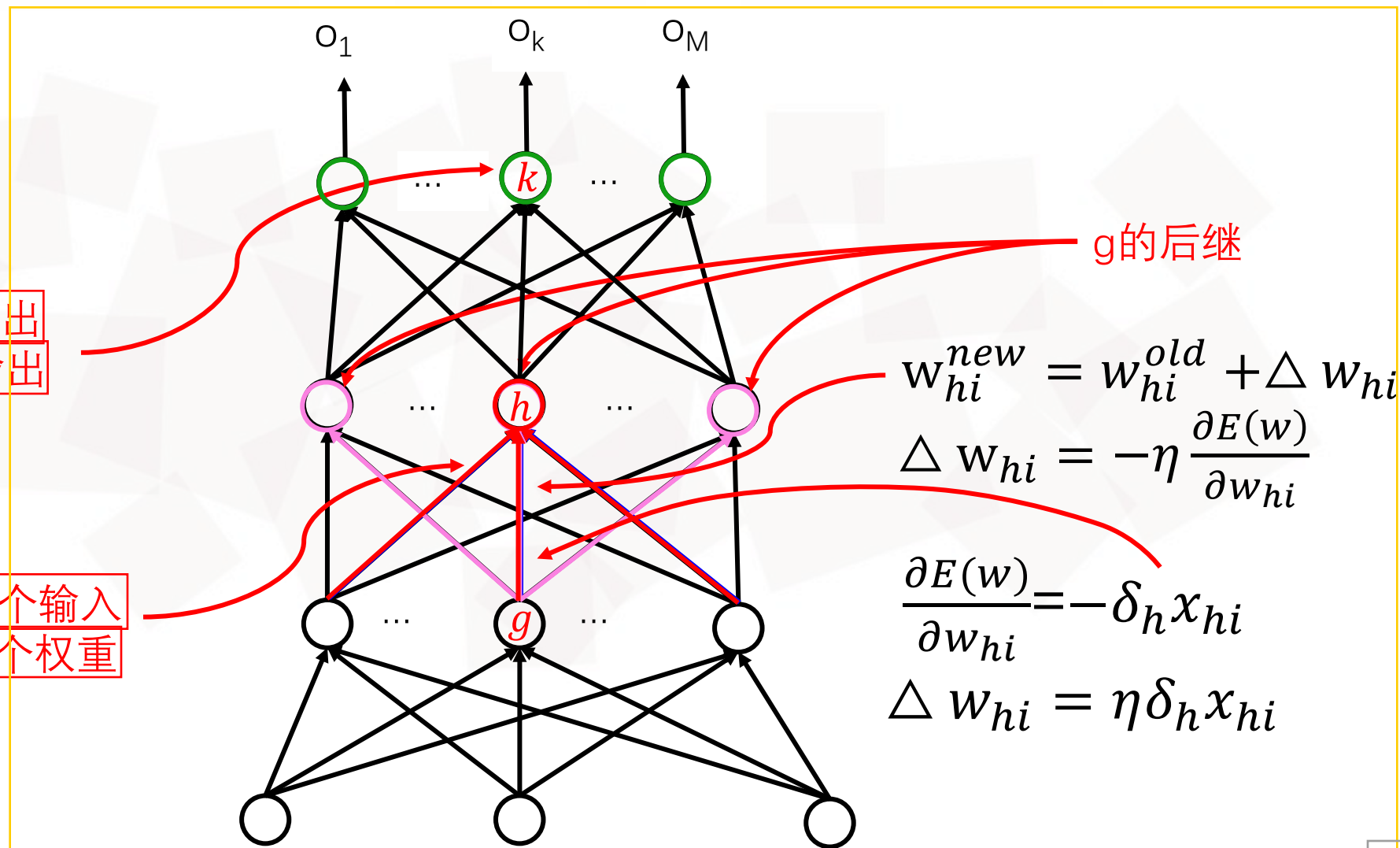
$$E_d(w) = \frac{1}{2} \sum_{k=1}^m (t_{kd} - o_{kd})^2$$

$$E(w) = \frac{1}{2} \sum_{d=k}^{k+B} \sum_{k=1}^m (t_{kd} - o_{kd})^2$$

符号说明

t_k : k的希望输出
 o_k : k的实际输出

x_{hi} : h的第i个输入
 w_{hi} : h的第i个权重

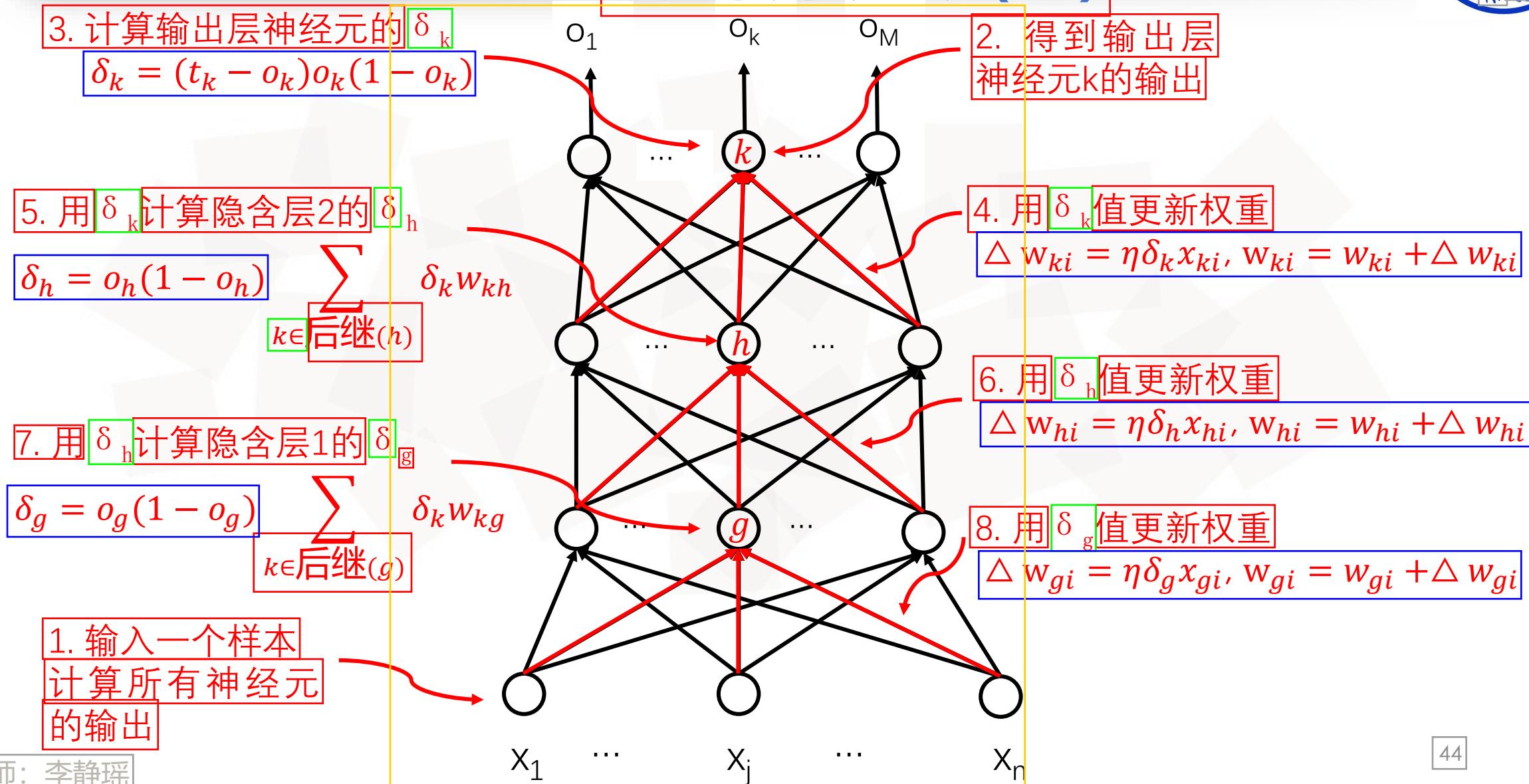




反向传播算法 (BP: Back Propagation)

- 又称作误差反向传播算法
- 给出了一种计算偏导数的方法

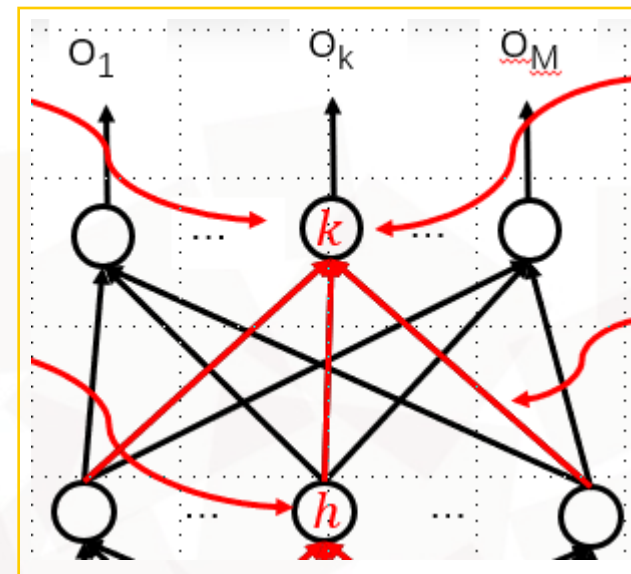
反向传播算法 (BP)



$$o_k = \text{Sigmoid}(\sum_i (w_{ki} \cdot x_{ki}) + b_k)$$

$$\text{Loss} = \frac{1}{2} (t_k - o_k)^2$$

$$\begin{aligned} \frac{\partial \text{Loss}}{\partial w_{ki}} &= -(t_k - o_k) \cdot \frac{\partial o_k}{\partial w_{ki}} \\ &= -(t_k - o_k) \cdot o_k \cdot (1 - o_k) \cdot \frac{\partial (\sum_i (w_{ki} \cdot x_{ki}) + b_k)}{\partial w_{ki}} \\ &= -(t_k - o_k) \cdot o_k \cdot (1 - o_k) \cdot x_{ki} = -\delta_k x_{ki} \end{aligned}$$



神经网络的训练

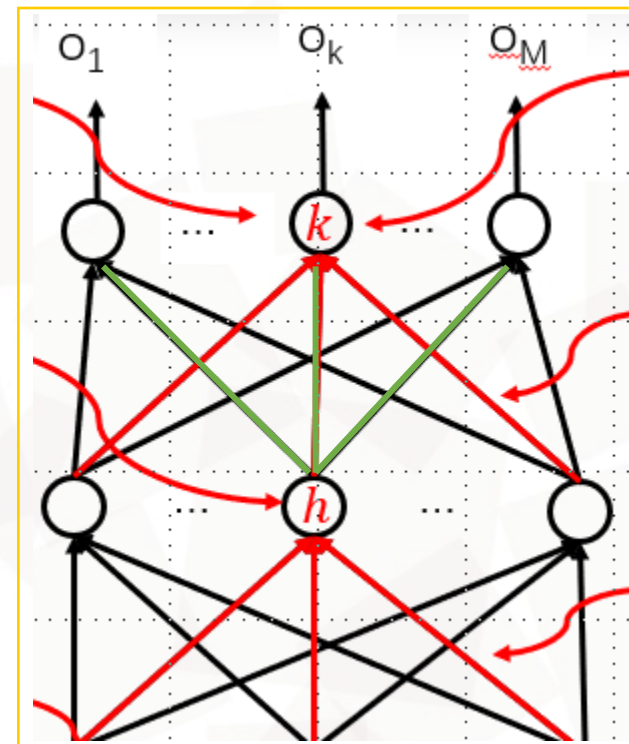


$$o_k = \text{Sigmoid} \left((w_{kh} \cdot o_h) + \sum_{h' \neq h} (w_{kh'} \cdot o_{h'}) + b_k \right)$$

$$o_h = \text{Sigmoid} \left(\sum_i (w_{hi} \cdot x_{hi}) + b_h \right)$$

$$\text{Loss} = \frac{1}{2} \sum_{k=1}^M (t_k - o_k)^2$$

$$\begin{aligned} \frac{\partial \text{Loss}}{\partial w_{hi}} &= - \sum_{k=1}^M (t_k - o_k) \cdot \frac{\partial o_k}{\partial w_{hi}} \\ &= - \sum_{k=1}^M (t_k - o_k) \cdot o_k \cdot (1 - o_k) \cdot \frac{\partial (w_{kh} \cdot o_h)}{\partial w_{hi}} \\ &= - \sum_{k=1}^M (t_k - o_k) \cdot o_k \cdot (1 - o_k) \cdot w_{kh} \cdot \frac{\partial (o_h)}{\partial w_{hi}} \end{aligned}$$



$$\begin{aligned}
 o_h &= \text{Sigmoid} \left(\sum_i (w_{hi} \cdot x_{hi}) + b_h \right) \\
 \frac{\partial \text{Loss}}{\partial w_{hi}} &= - \sum_{k=1}^M (t_k - o_k) \cdot \frac{\partial o_k}{\partial w_{hi}} \\
 &= - \sum_{k=1}^M (t_k - o_k) \cdot o_k \cdot (1 - o_k) \cdot \frac{\partial (w_{kh} \cdot o_h)}{\partial w_{hi}} \\
 &= - \sum_{k=1}^M (t_k - o_k) \cdot o_k \cdot (1 - o_k) \cdot w_{kh} \cdot \frac{\partial (o_h)}{\partial w_{hi}} \\
 &= - \sum_{k=1}^M \underbrace{(t_k - o_k) \cdot o_k \cdot (1 - o_k) \cdot w_{kh} \cdot o_h \cdot (1 - o_h)}_{\delta_k \cdot w_{kh} \cdot o_h \cdot (1 - o_h)} \cdot x_{hi} \\
 &= - \sum_{k=1}^M \delta_k \cdot w_{kh} \cdot o_h \cdot (1 - o_h) \cdot x_{hi} = -\delta_h x_{hi}
 \end{aligned}$$



该BP算法的条件

- 具体条件不同，算法会有差异，总体思路一样
- 全连接网络
 - 其它形式的网络
- 随机梯度下降算法
 - 批量、小批量梯度下降算法
- 激活函数：Sigmoid函数
 - 其它的激活函数
- 损失函数：误差平方和
 - 其它的损失函数



交叉熵损失函数

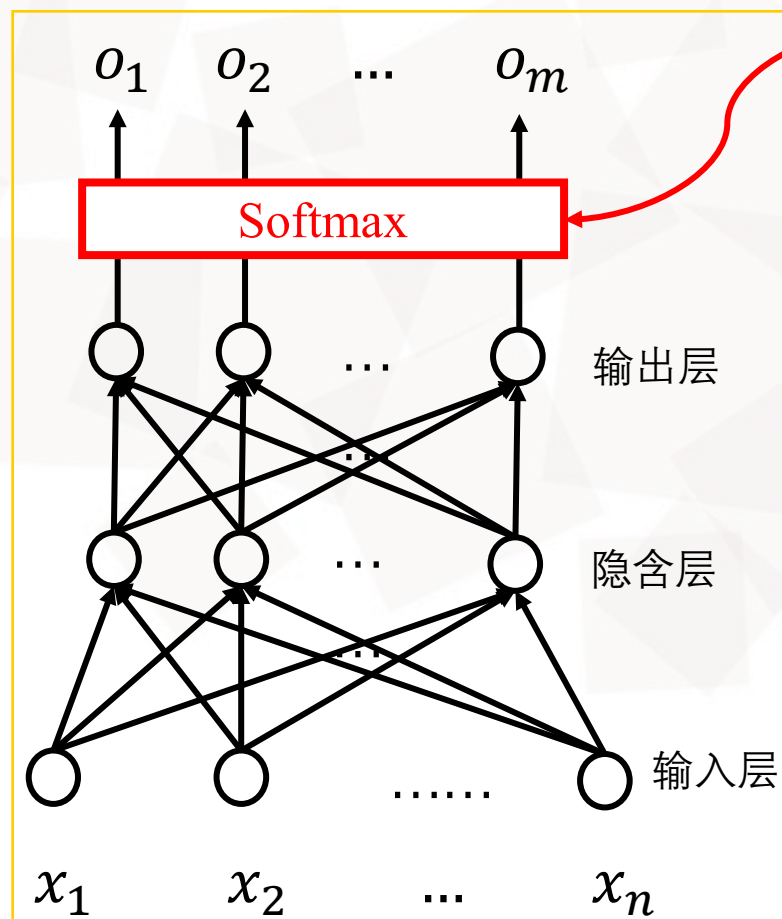
$$H_d(w) = -\sum_{k=1}^M t_{kd} \log(o_{kd})$$

$$H(w) = \sum_{d=1}^N H_d(w) = -\sum_{d=1}^N \sum_{k=1}^M t_{kd} \log(o_{kd})$$

t_{kd} : 样本d对应的希望输出值

o_{kd} : 样本d对应的实际输出值, 要求 o_{kd} 是个概率值

交叉熵损失函数



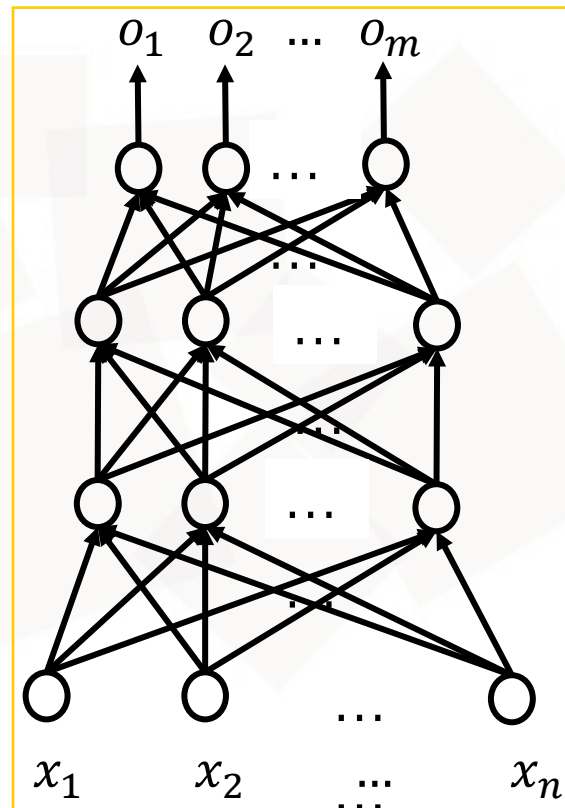
$$o_k = \frac{e^{net_k}}{\sum_{i=1}^m e^{net_i}}$$

$$0 \leq o_k \leq 1$$
$$\sum_{k=1}^m o_k = 1$$

交叉熵损失函数

- 对于分类问题
- 对于给定输入样本 d ，只有 d 对应的希望输出为1，其他为0

$$H_d(w) = -\log(o_{kd})$$





两种损失函数的作用

- 误差平方和损失函数
 - 用于输出是具体数值的问题
- 交叉熵损失函数
 - 用于分类问题

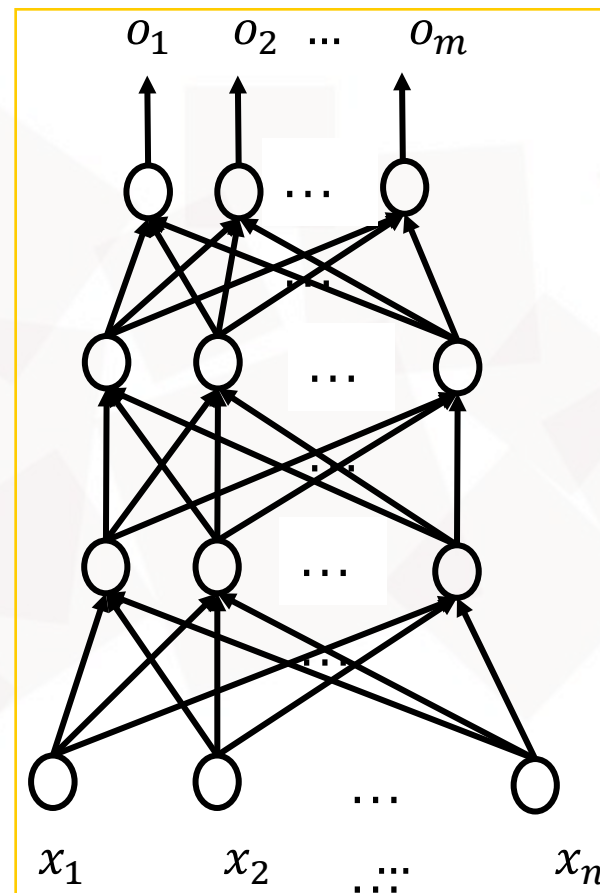


神经网络的训练流程

- 建立数据集
- 损失函数
 - 误差平方和、交叉熵
- 梯度下降法
 - 批量梯度下降算法
 - 随机梯度下降算法
 - 小批量梯度下降算法
- BP算法
 - 随机梯度下降算法的一种实现

全连接网络的不足

- 连接权重过多
- 影响训练速度
- 影响使用速度



提取局部模式

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	1	1	1	0	0	0	0	0	0	0	0	0	0	1	1	1	0
0	1	1	1	0	0	0	0	0	0	0	0	0	0	1	1	1	0
0	1	1	1	0	0	0	0	0	0	0	0	0	0	1	1	1	0
0	1	1	1	0	0	0	0	0	0	0	0	0	0	1	1	1	0
0	1	1	1	0	0	0	0	0	0	0	0	0	0	1	1	1	0
0	1	1	1	0	0	0	0	0	0	0	0	0	0	1	1	1	0
0	1	1	1	0	0	0	0	0	0	0	0	0	0	1	1	1	0
0	1	1	1	0	0	0	0	0	0	0	0	0	0	1	1	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

边缘信息

输入

2	1	0	2	3
9	5	4	2	0
2	3	4	5	6
1	2	3	1	0
0	4	4	2	8

权重 (模式)

-1	0	1
-1	0	1
-1	0	1

输出 (匹配结果)

-5	0	1
-1	-2	-5
8	-1	3

$$\begin{aligned} & (-1) \times 2 + 0 \times 1 + 1 \times 0 \\ & + (-1) \times 9 + 0 \times 5 + 1 \times 4 \\ & + (-1) \times 2 + 0 \times 3 + 1 \times 4 = -5 \end{aligned}$$

输入

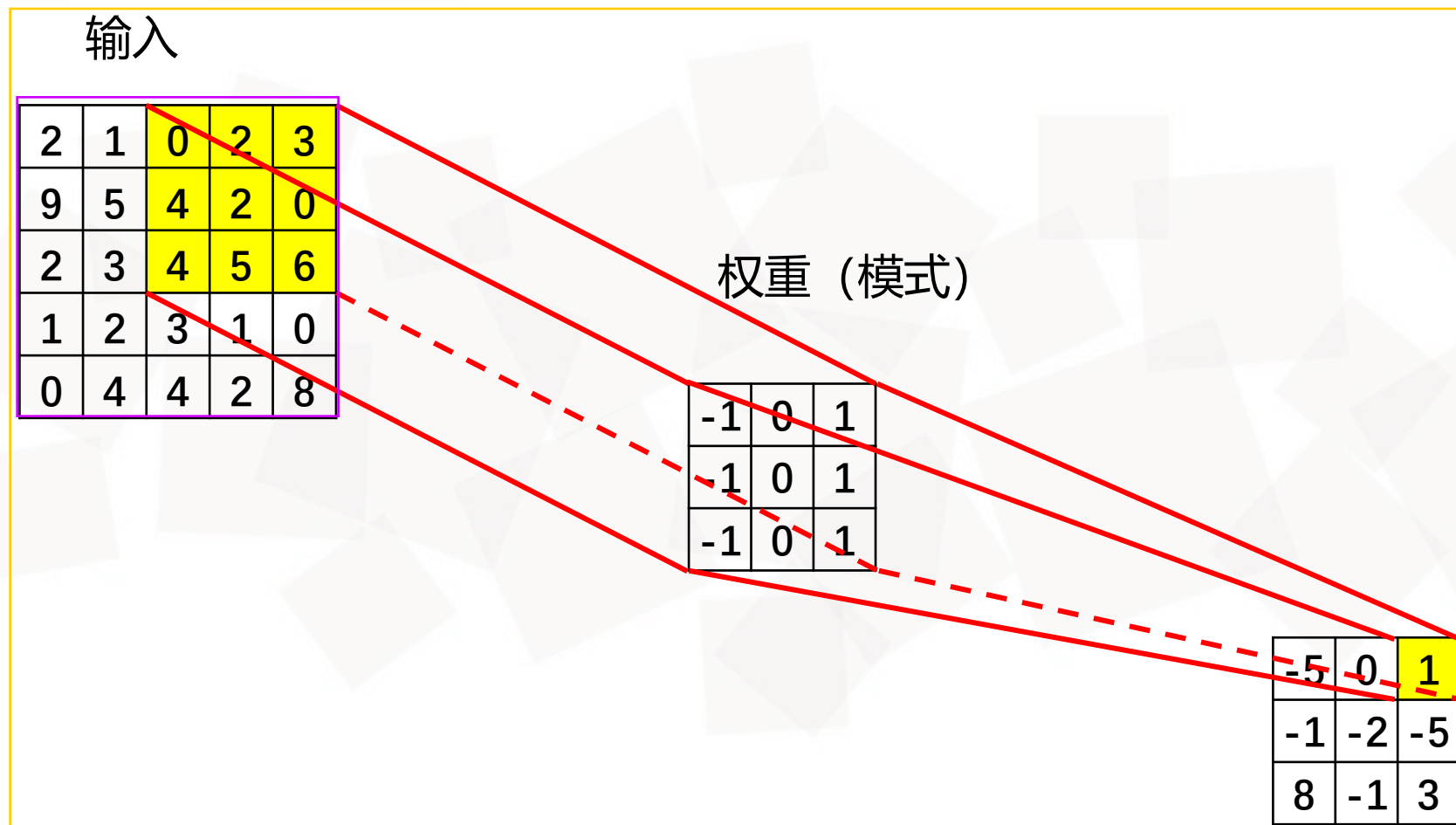
2	1	0	2	3
9	5	4	2	0
2	3	4	5	6
1	2	3	1	0
0	4	4	2	8

权重 (模式)

-1	0	1
-1	0	1
-1	0	1

输出 (匹配结果)

-5	0	1
-1	-2	-5
8	-1	3



输入

2	1	0	2	3
9	5	4	2	0
2	3	4	5	6
1	2	3	1	0
0	4	4	2	8

权重 (模式)

-1	0	1
-1	0	1
-1	0	1

卷积核
训练获得

输出 (匹配结果)

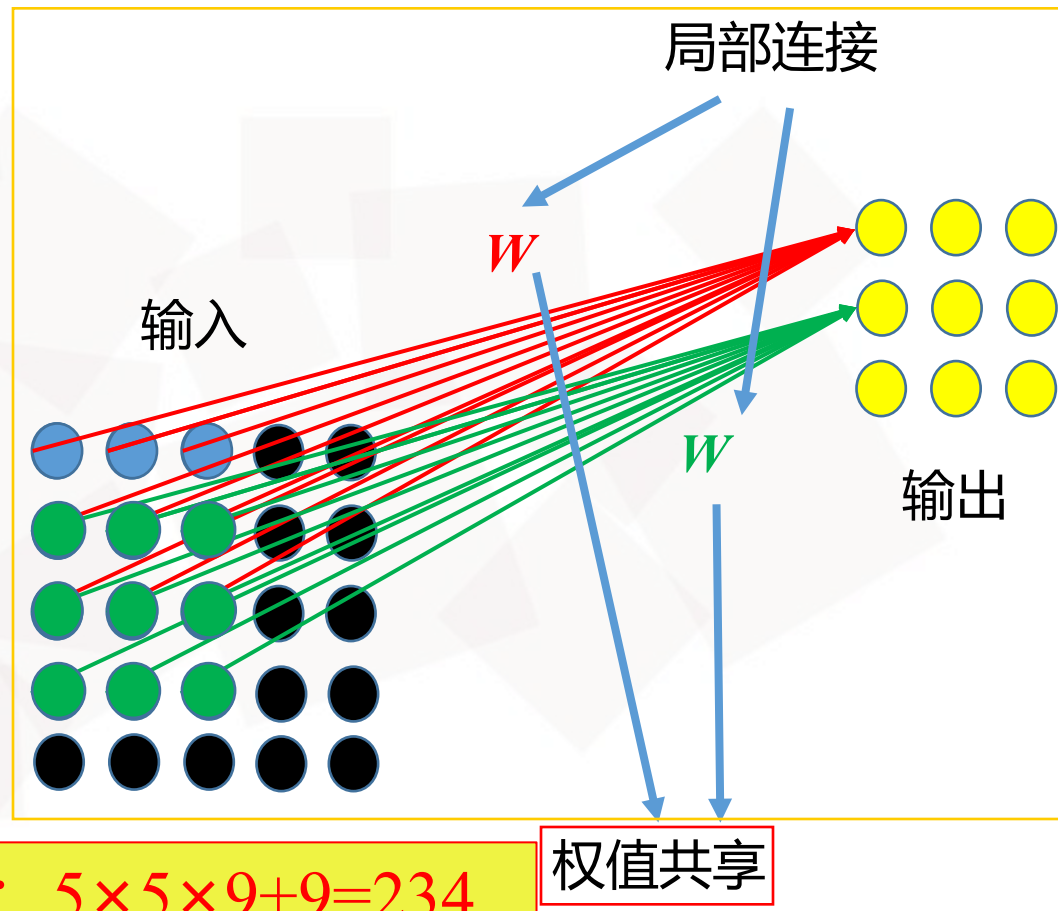
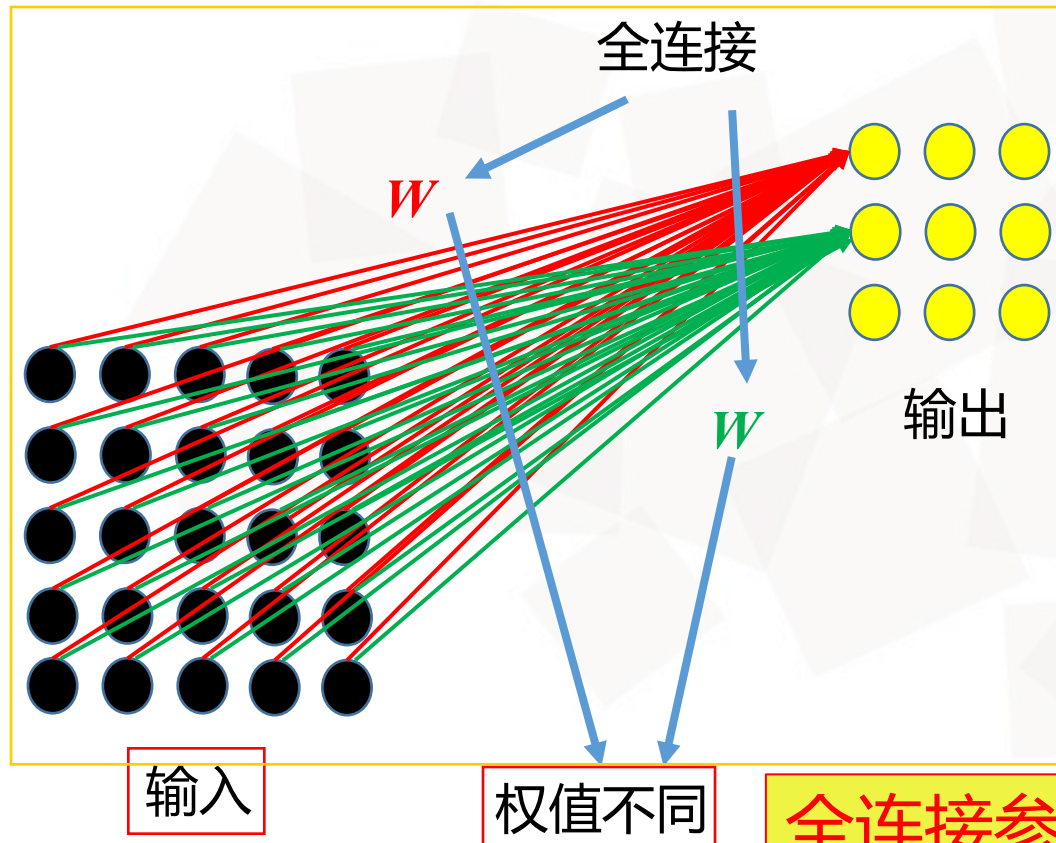
5	0	1
-1	-2	-5
8	-1	3

卷积神经网络

-局部连接

-权值共享

全连接、卷积神经网络对比



全连接参数: $5 \times 5 \times 9 + 9 = 234$

卷积核参数: $3 \times 3 + 1 = 10$

例：边缘提取

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

输入

正边缘

3	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	3
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	-1	-3	-4	-4	-4	-4	-4	-4	-4	-4	-3	-1	0	0	0
0	0	-1	-3	-4	-4	-4	-4	-4	-4	-4	-4	-3	-1	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	3	4	4	4	4	4	4	4	4	3	1	0	0	0
0	0	1	3	4	4	4	4	4	4	4	4	3	1	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
-3	-4	-4	-4	-4	-4	-4	-4	-4	-4	-4	-4	-4	-4	-4	-4	-3

输出

-1	-2	-1
0	0	0
1	2	1

卷积核

例：边缘提取

[illegible]

图像

[illegible]

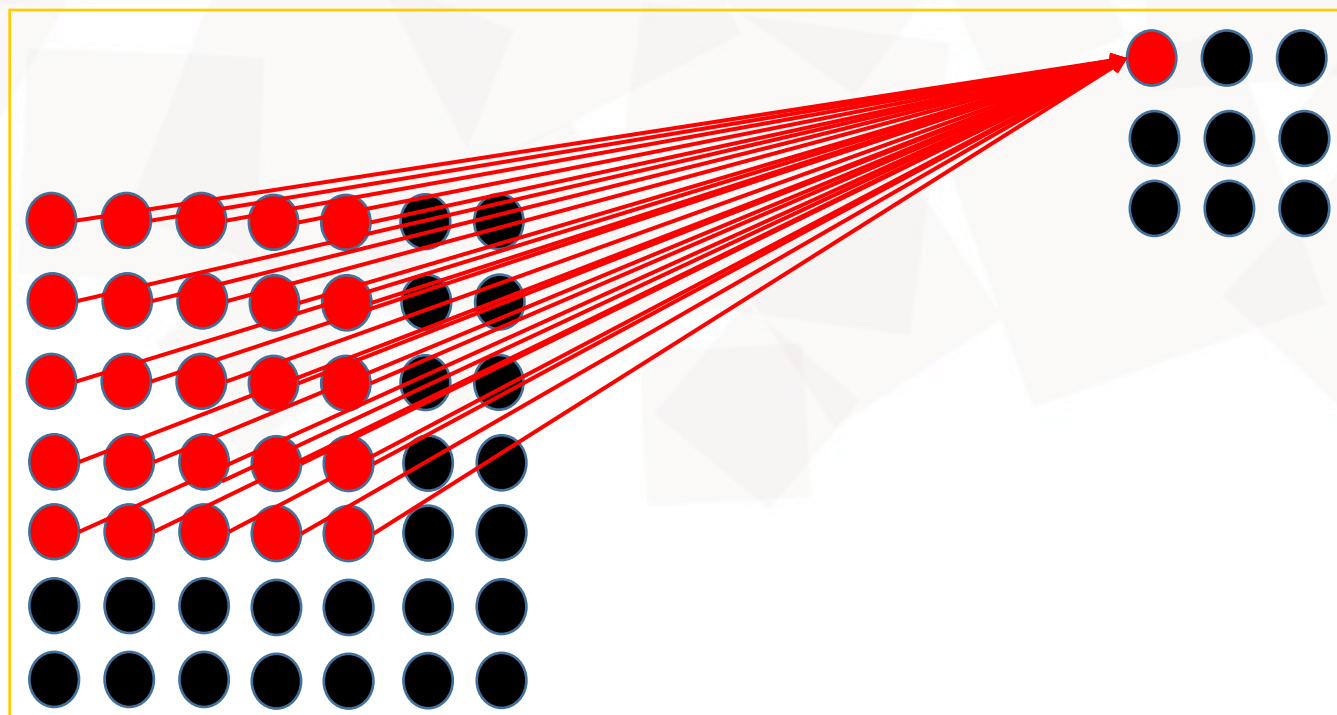
匹配结果

[illegible]

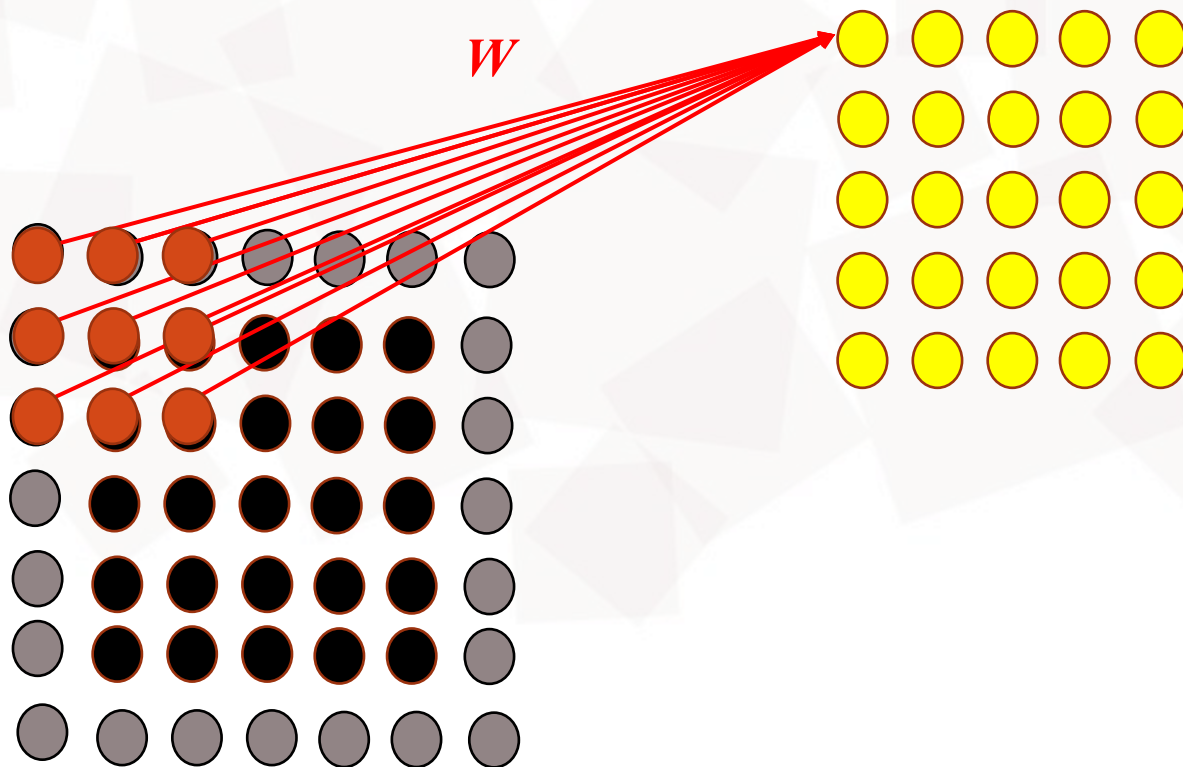
Sigmoid

卷积核的大小

- 对于二维输入，一般是 3×3 、 5×5 、 7×7 ...

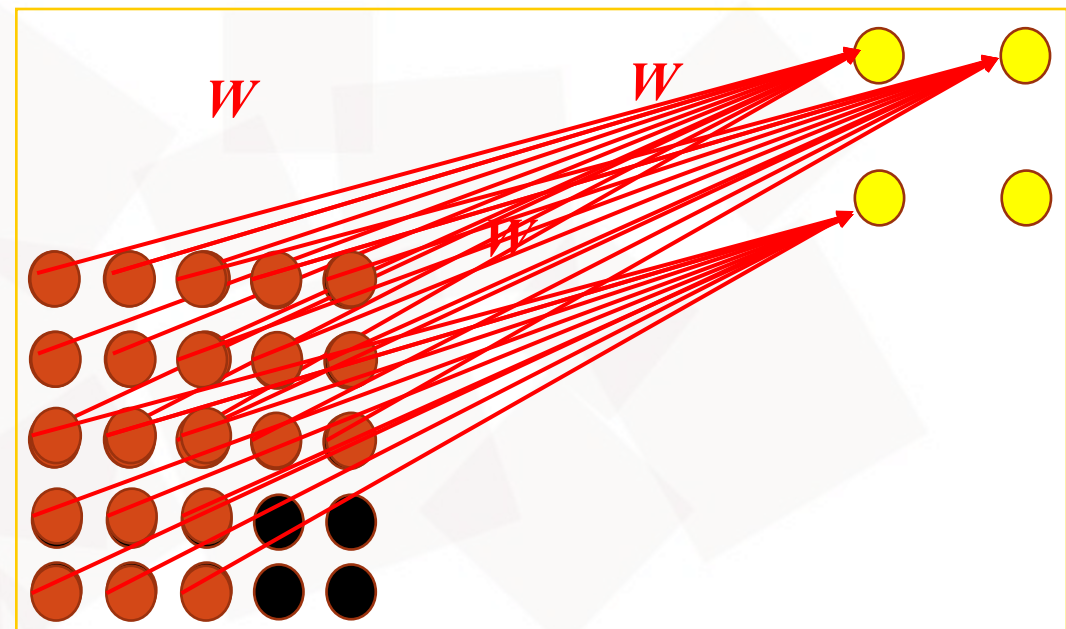


填充

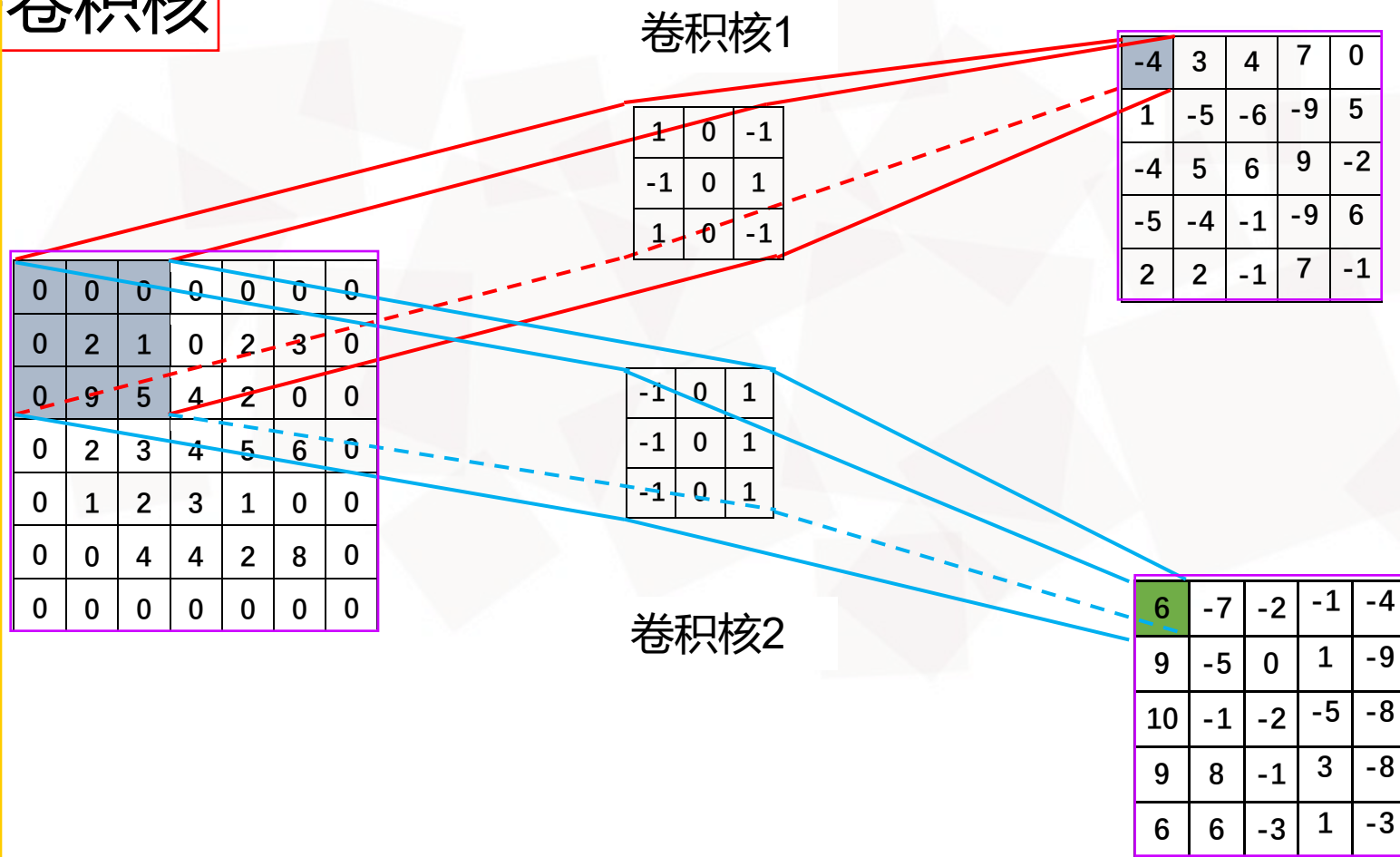


步长

- 卷积核每次移动的距离
- 步长是可以设定的 (以2为例)



多卷积核



输出 (通道1)

$$1 \times 0 + 0 \times 0 + (-1) \times 0$$

$$+ (-1) \times 0 + 0 \times 2 + 1 \times 1$$

$$+ 1 \times 0 + 0 \times 9 + (-1) \times 5 = -4$$

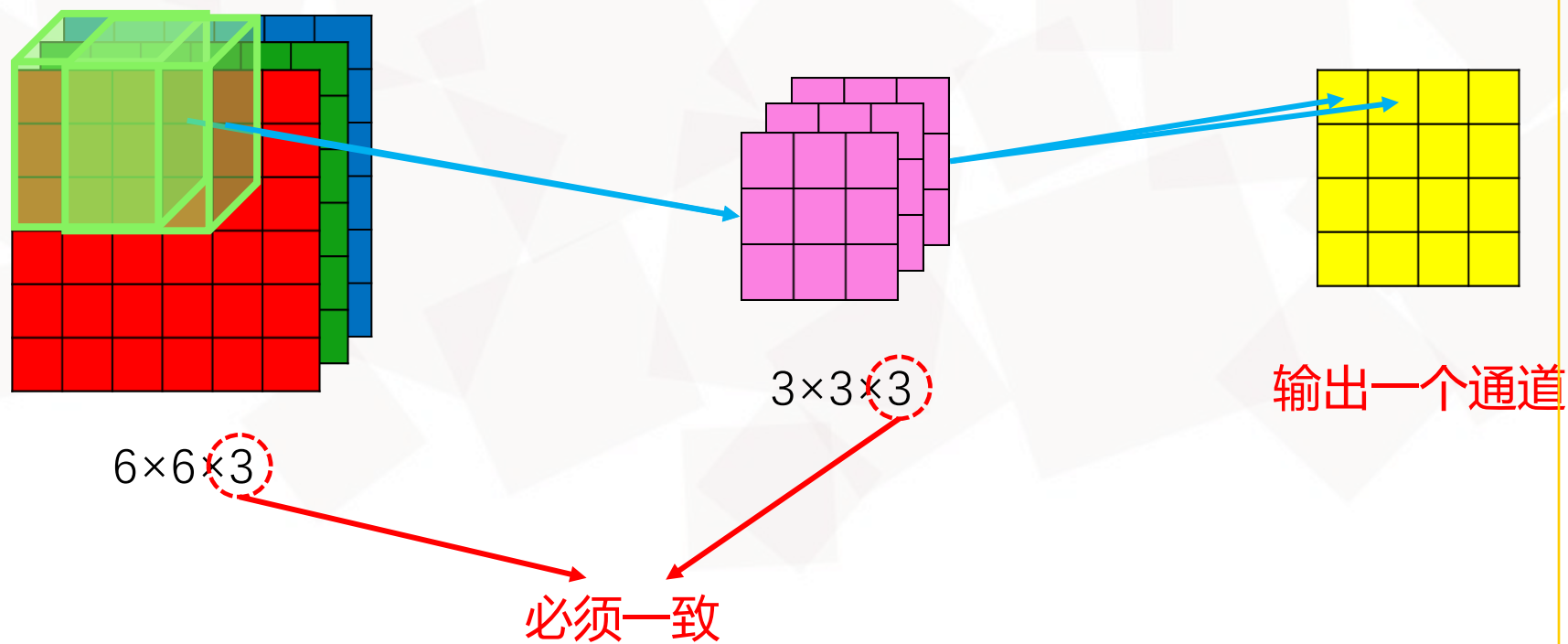
输出 (通道2)

$$(-1) \times 0 + 0 \times 0 + 1 \times 0$$

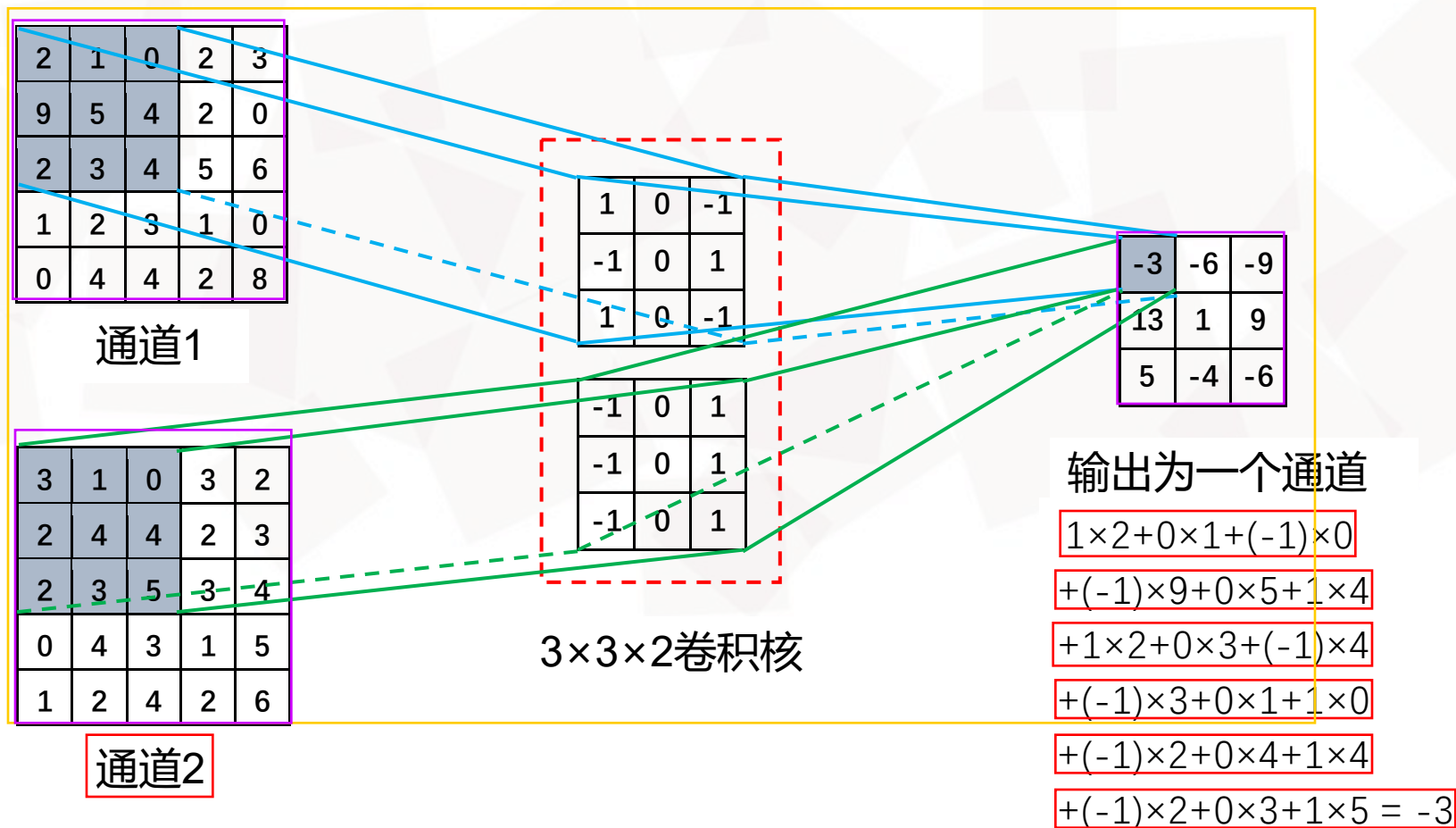
$$+ (-1) \times 0 + 0 \times 2 + 1 \times 1$$

$$+ (-1) \times 0 + 0 \times 9 + 1 \times 5 = 6$$

多通道输入时的卷积

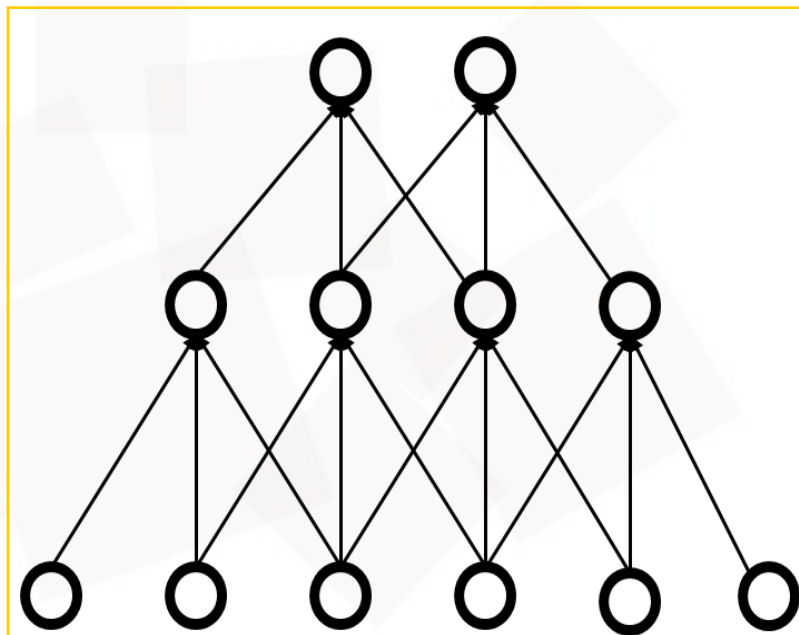


多通道卷积举例



卷积核的大小

- 小卷积核：细粒度特征
- 大卷积核：大粒度特征
- 多层小卷积实现大卷积
 - 两层 3×3 卷积等效一个 5×5 的卷积



池化

- 一种降维的手段

0	0	1	1	1	1	1	1	1	1	1	1	1	0	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	0	0	0	0	0	0	0	0	0	0	1	1
1	1	1	0	0	0	0	0	0	0	0	0	0	1	1
1	1	1	0	0	0	0	0	0	0	0	0	0	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	1	1	1	1	1	1	1	1	1	1	1	0	0
0	1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	0	0	0	0	0	0	0	0	0	0	1	1
1	1	1	0	0	0	0	0	0	0	0	0	0	1	1
1	1	1	0	0	0	0	0	0	0	0	0	0	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	1	1	1	1	1	1	1	1	1	1	1	0	0

0	1	1	1	0
1	0	0	0	1
0	1	1	1	0
1	0	0	0	1
0	1	1	1	0

池化方法

- 最大池化
 - 取窗口内的最大值
 - 窗口大小、步长都可以设定
- 平均池化
 - 取窗口内的平均值

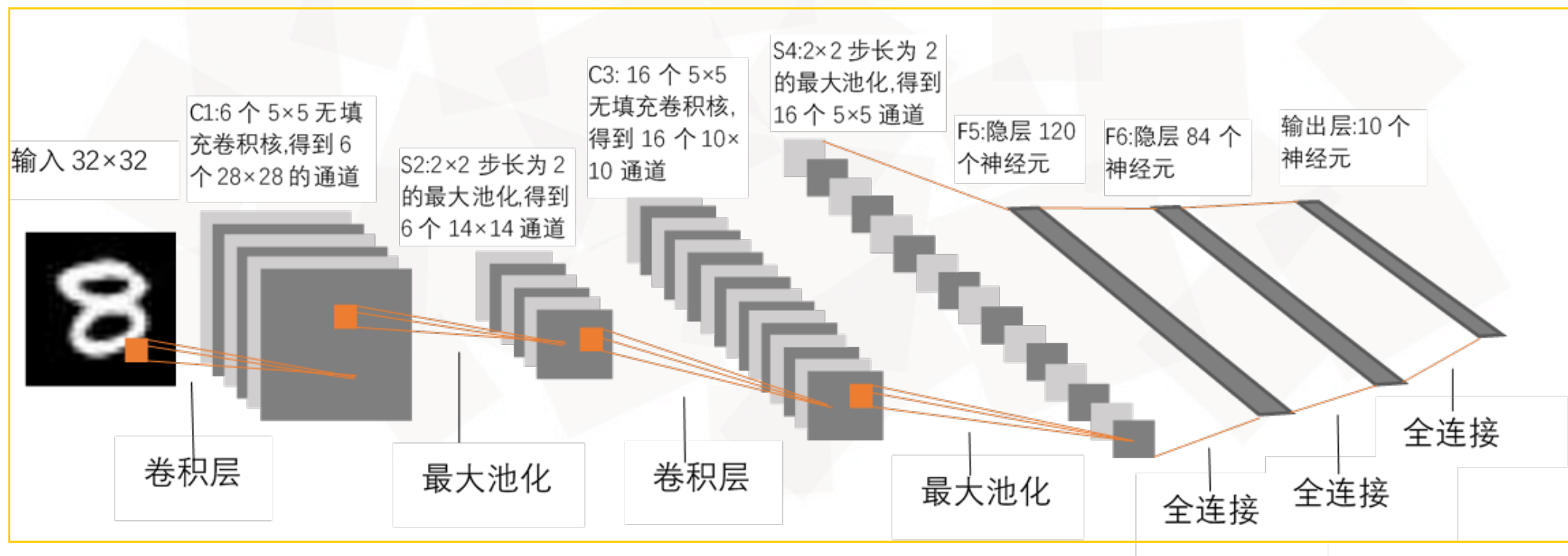
1	1	2	4	5	1
5	6	7	8	0	7
3	2	1	0	4	1
1	2	3	4	3	2
8	6	5	3	3	1
1	0	2	1	2	0

输入为一个通道

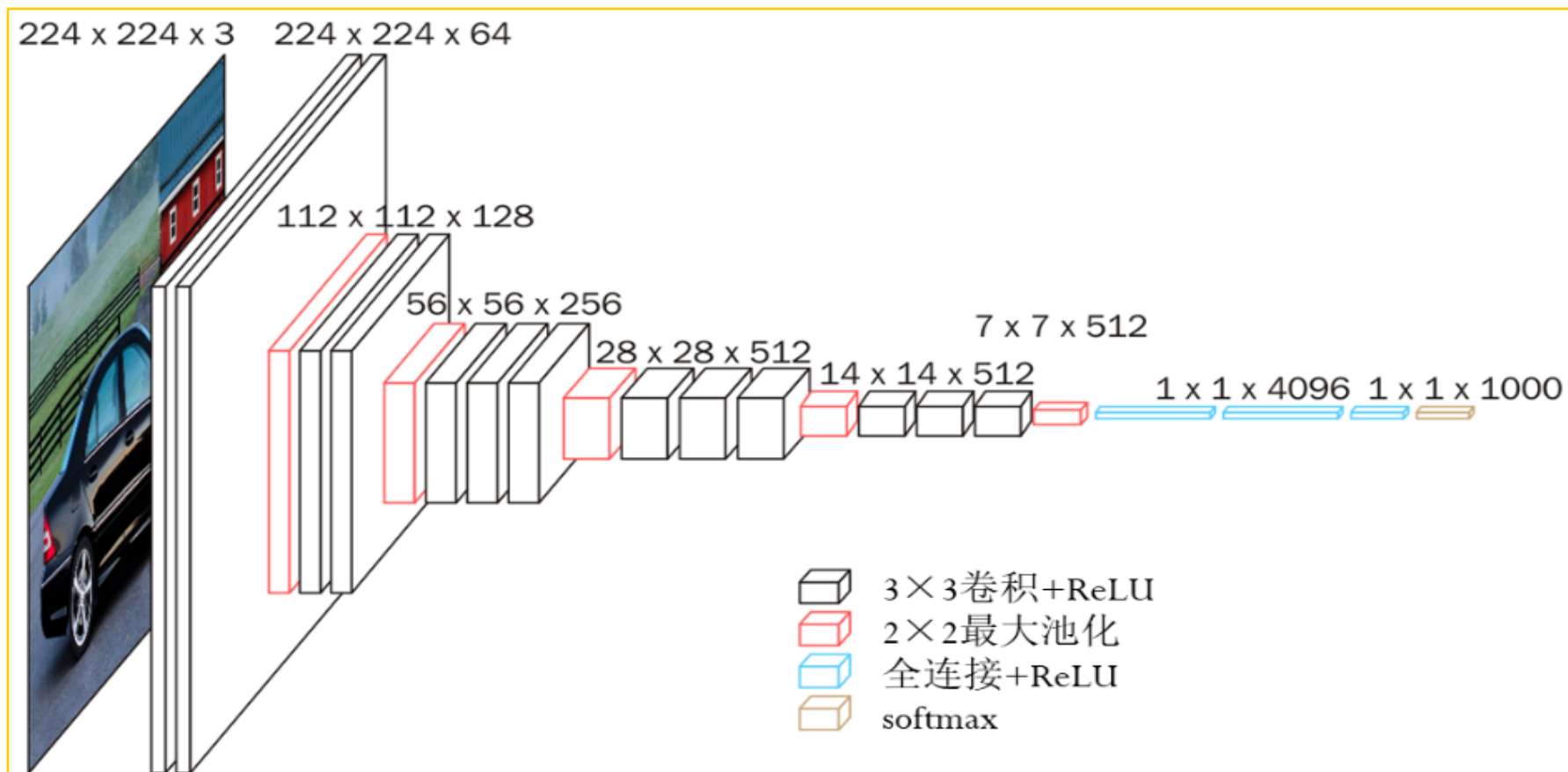
最大池化
窗口: 2×2
步长: 2

6	8	7
3	4	4
8	5	3

LeNet神经网络 (Yann LeCun, 1998)



VGG-16神经网络 (ImageNet)





神经网络训练的两大问题：

- 梯度消失问题
- 过拟合问题

什么是梯度消失问题：

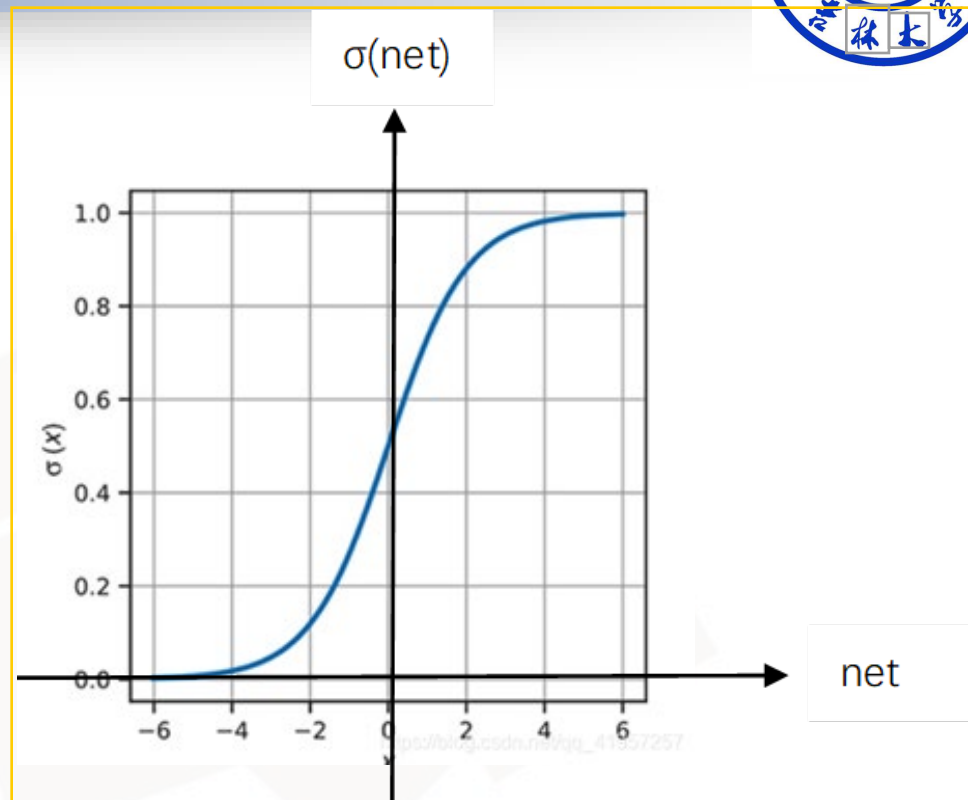
$$\delta_h = \boxed{o_h(1 - o_h)} \sum_{k \in \text{后续}(h)} \delta_k w_{kh} \leq 0.25$$

- 当神经网络层数增多之后

$$\delta_h \longrightarrow 0$$

- 梯度消失问题

$$\Delta w_{ji} = \eta \delta_j x_{ji}$$

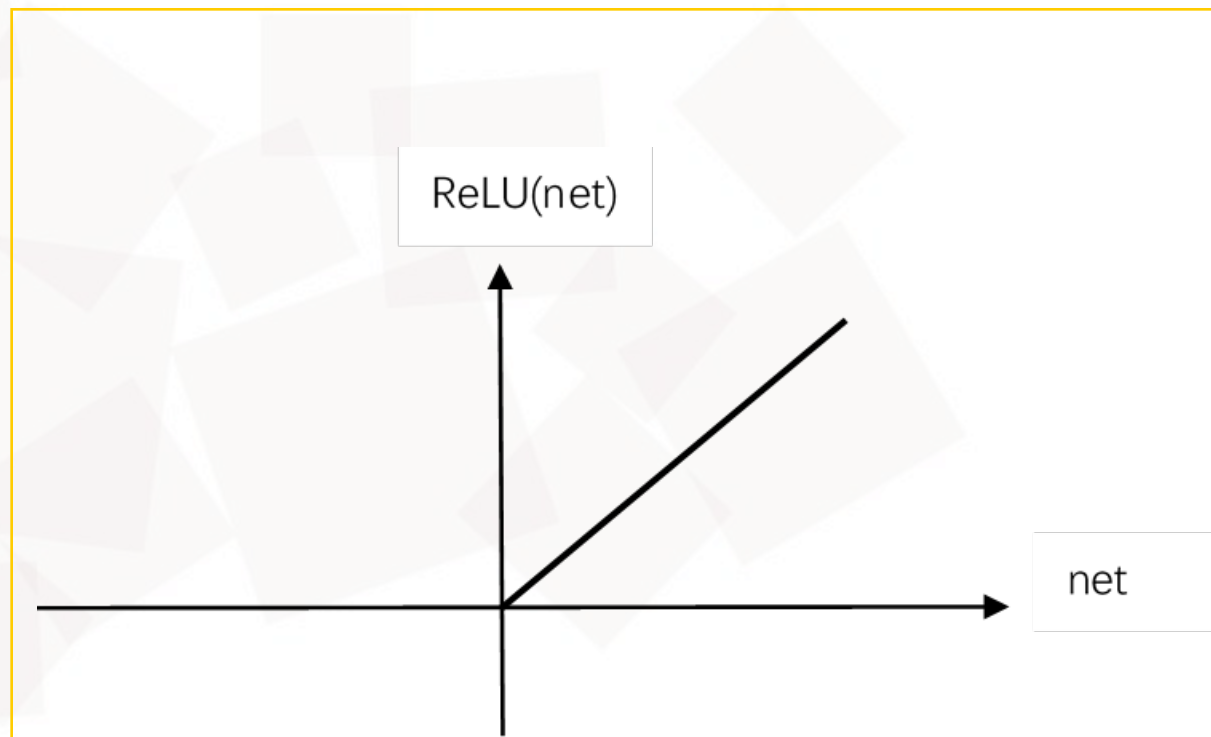


解决思路

- 使用ReLU激活函数

Sigmoid
的导数

$$\delta_h = o_h(1 - o_h) \sum_{k \in \text{后续}(h)} \delta_k w_{kh}$$



- ReLU的导数=1



两个解决梯度消失问题的网络结构

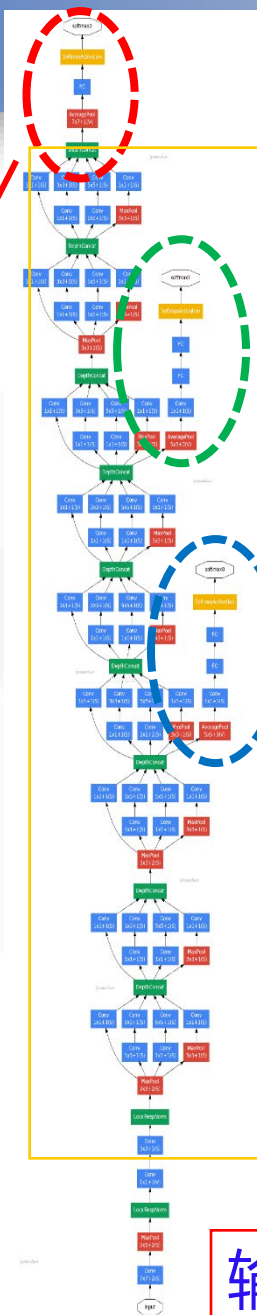
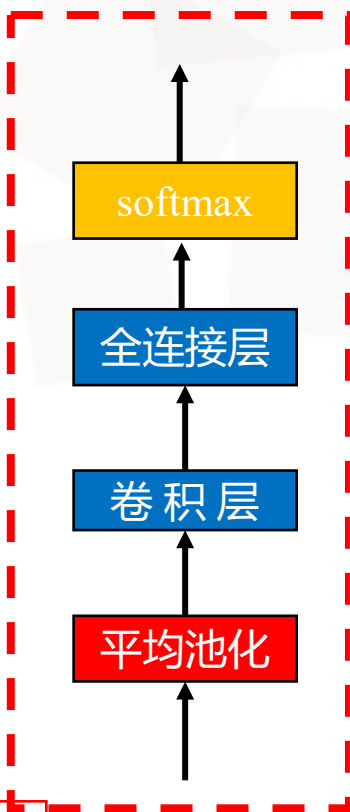
- GoogLeNet

- ResNet (残差网络)

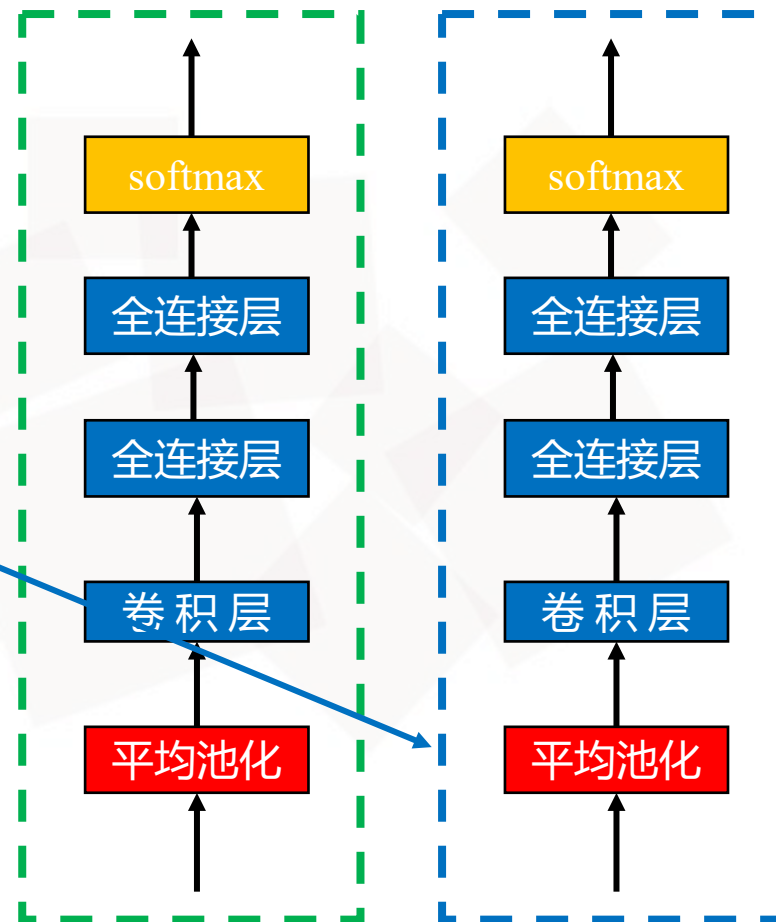
梯度消失问题



GoogLeNet



输入

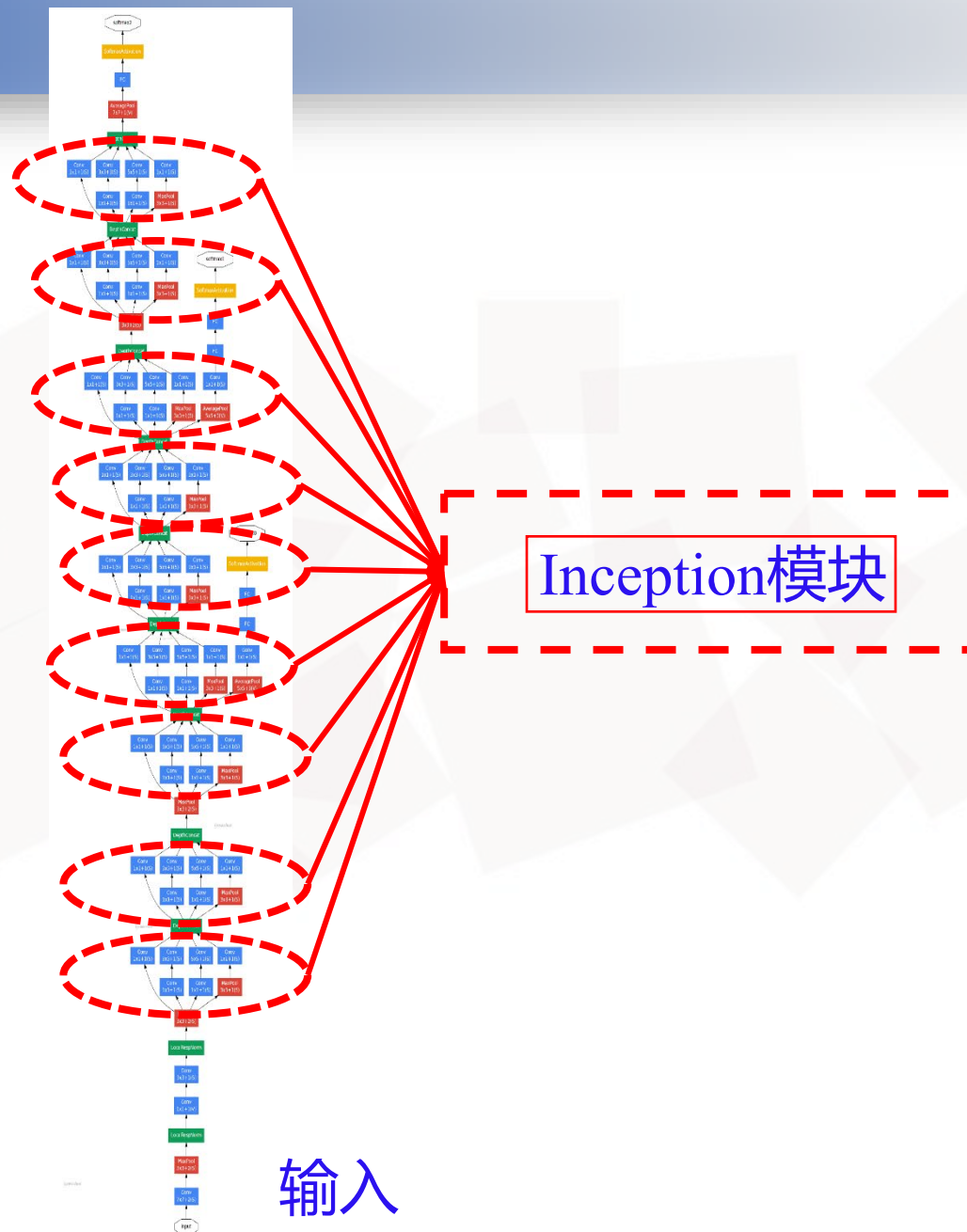


高楼供水系统



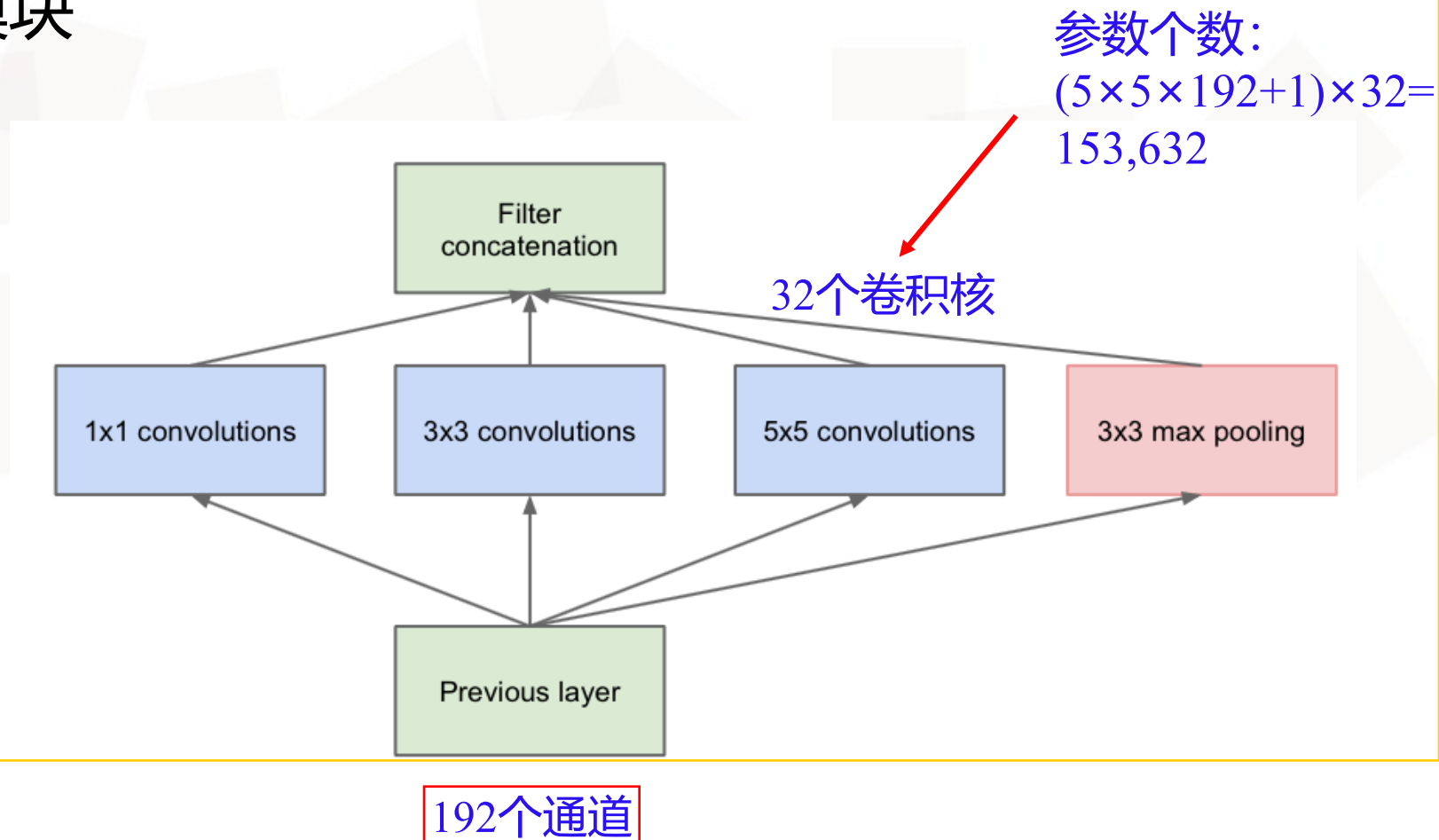
梯度消失问题

GoogLeNet



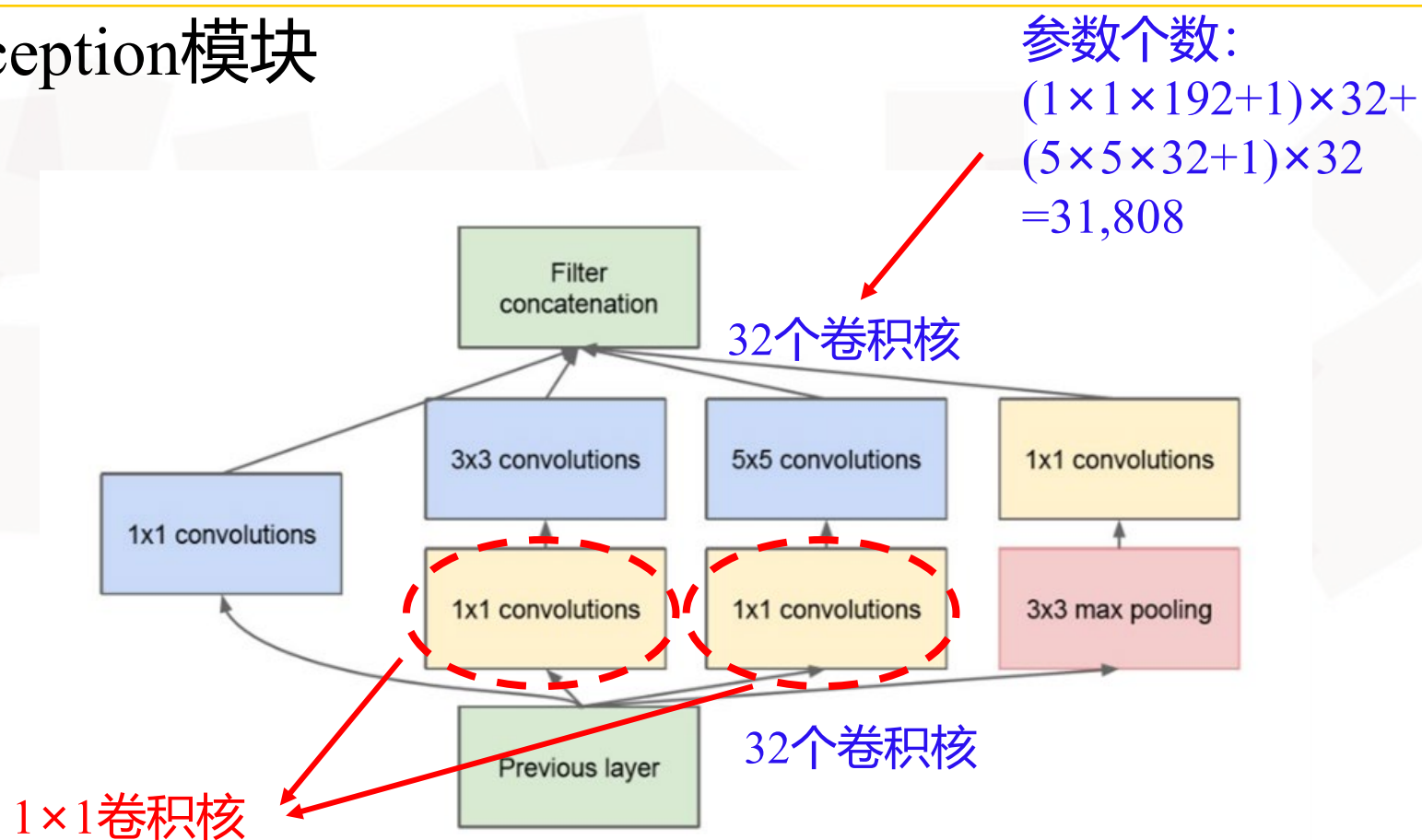


Inception模块

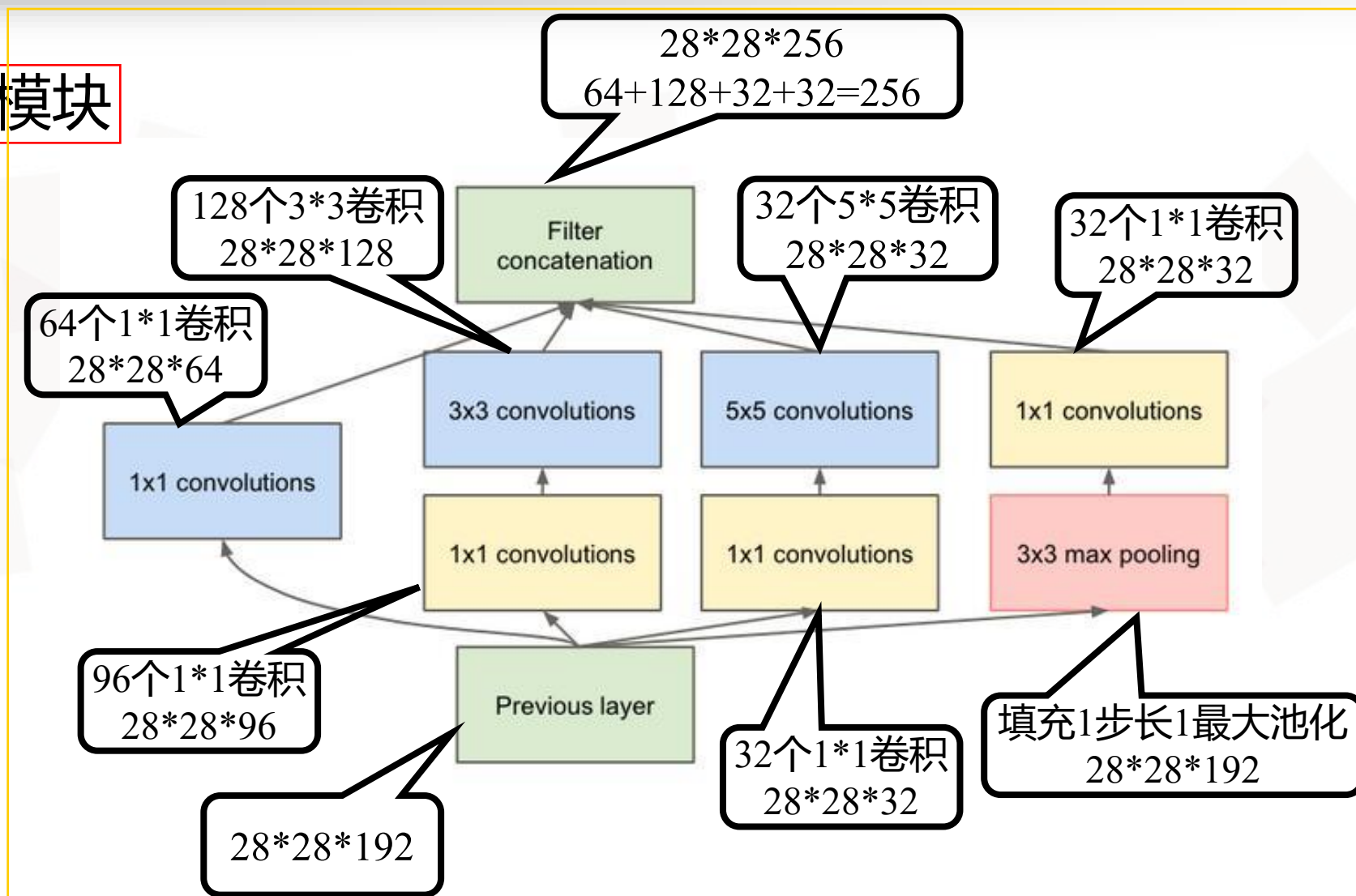




降维的Inception模块

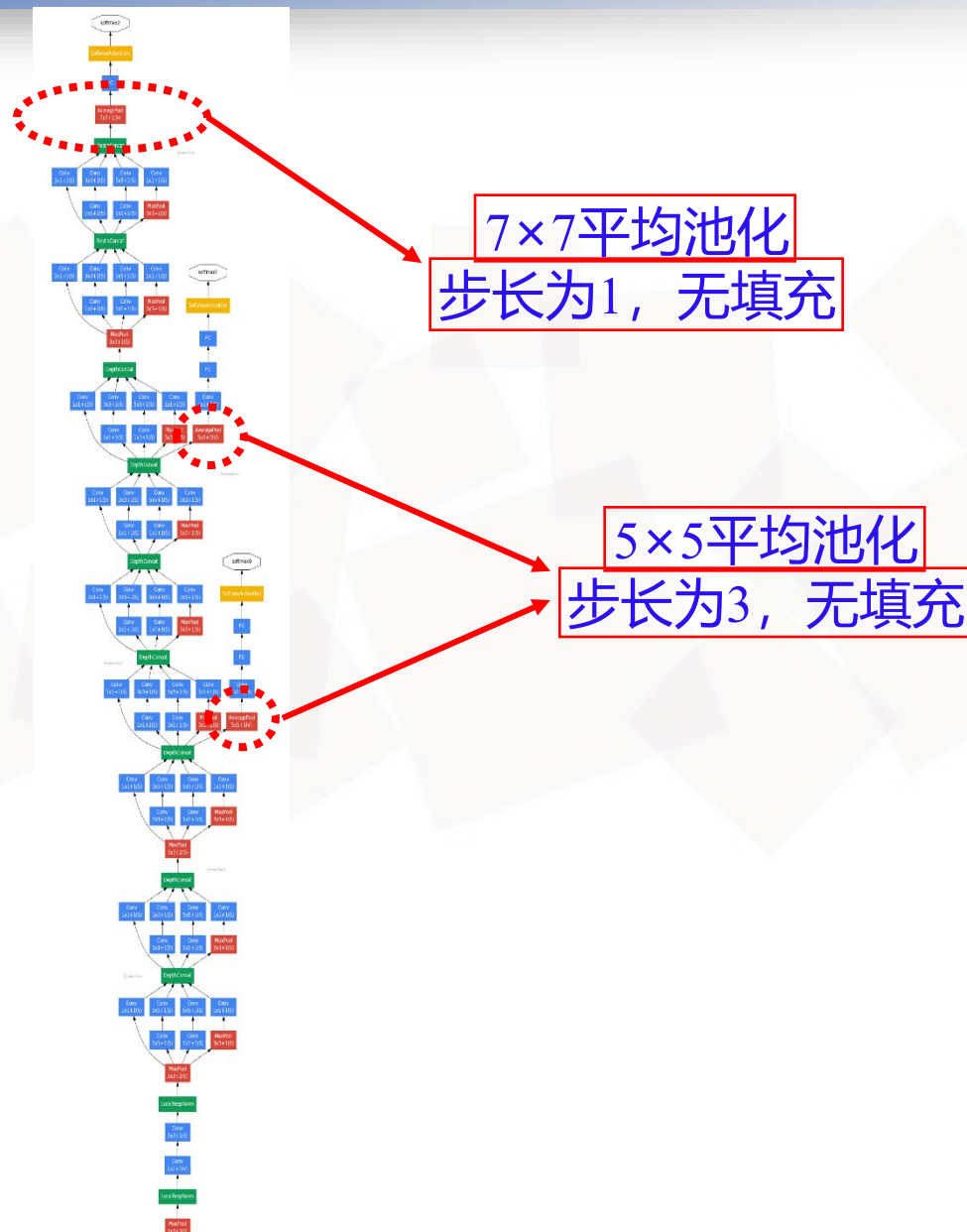


Inception模块



- 说明：每个Inception的参数有所不同，这是其中的一个

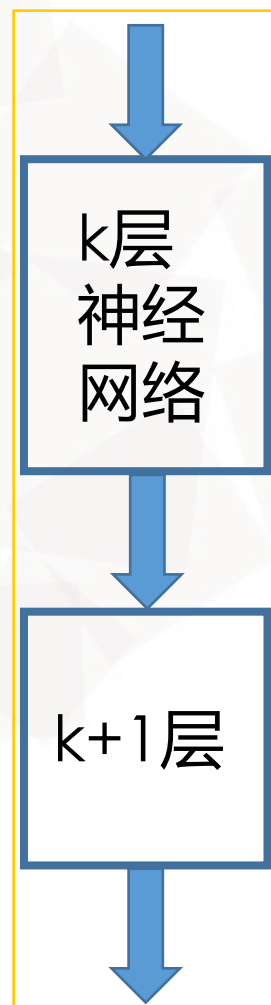
GoogLeNet





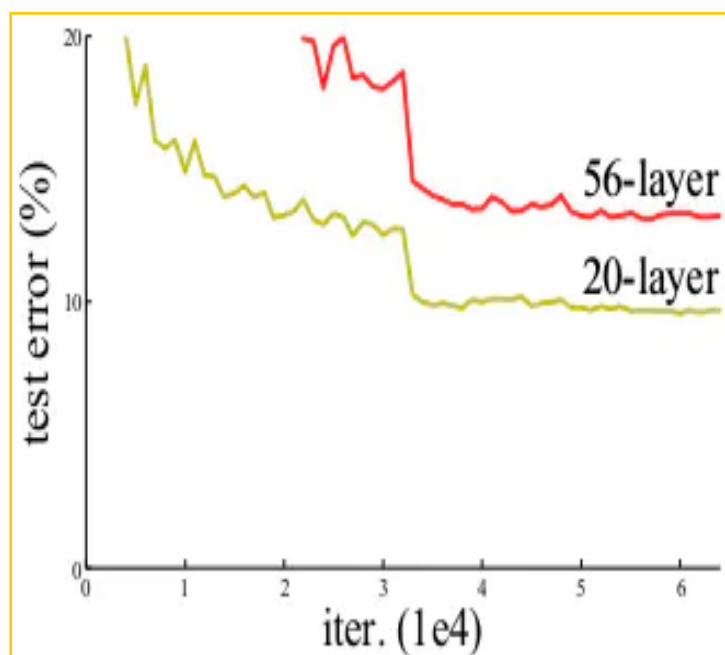
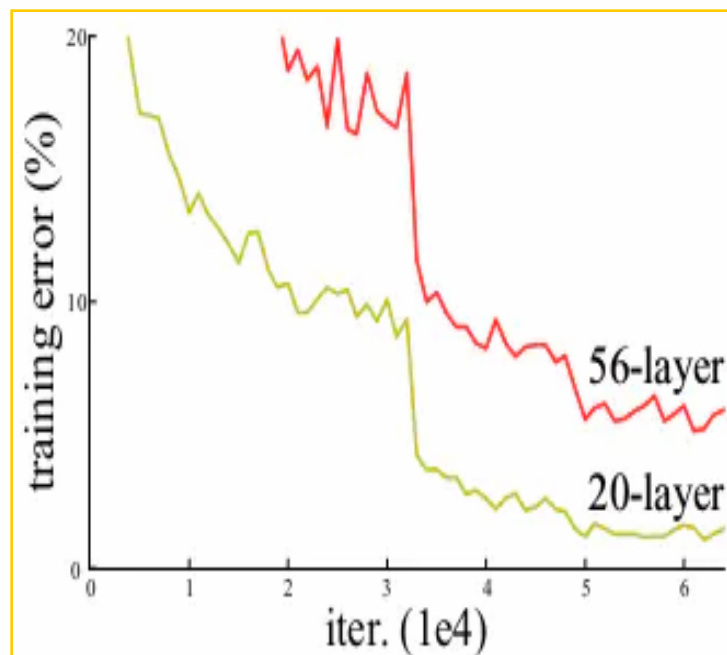
残差网络 (ResNet)

- 神经网络的退化现象



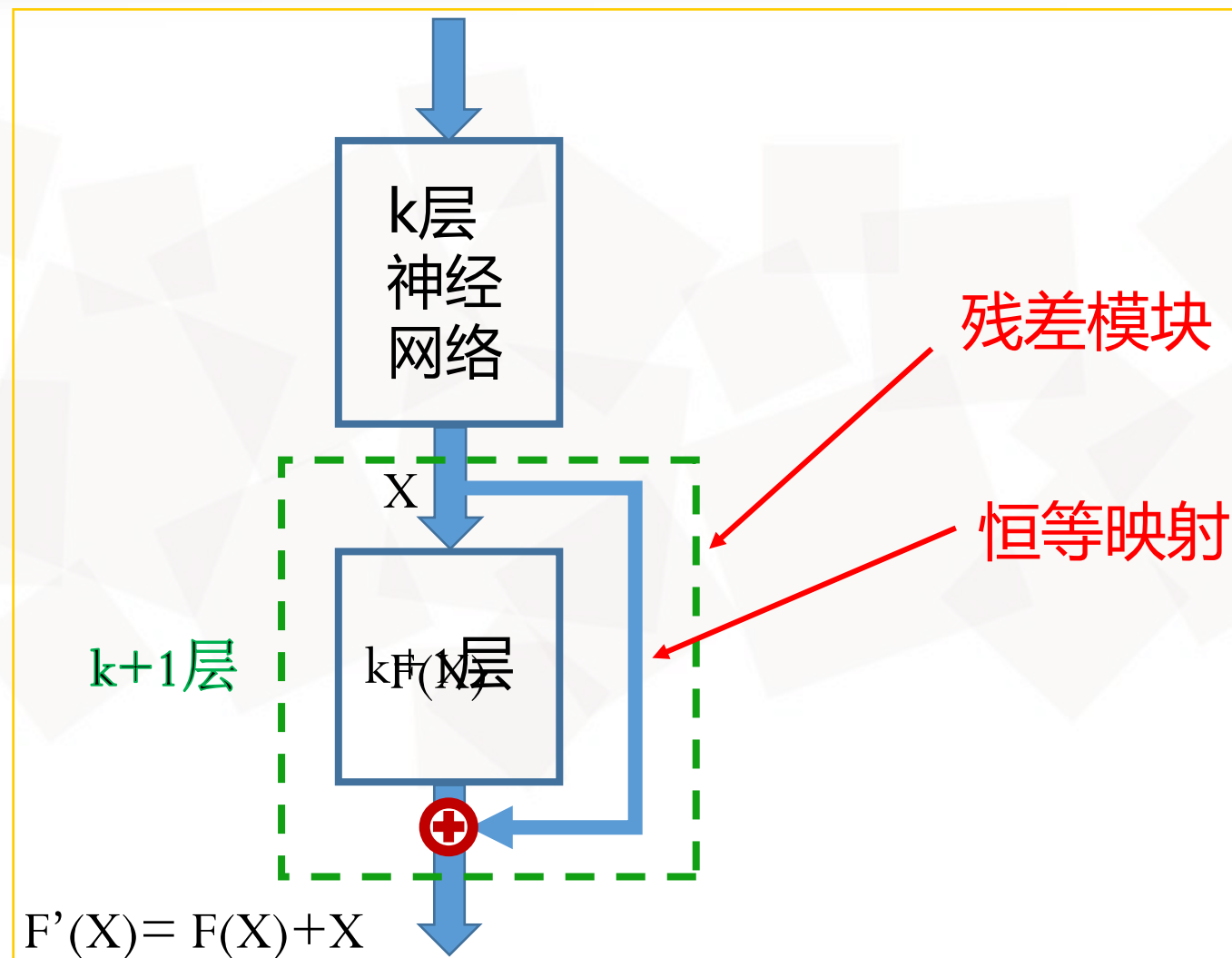
残差网络 (ResNet)

- 神经网络的退化现象

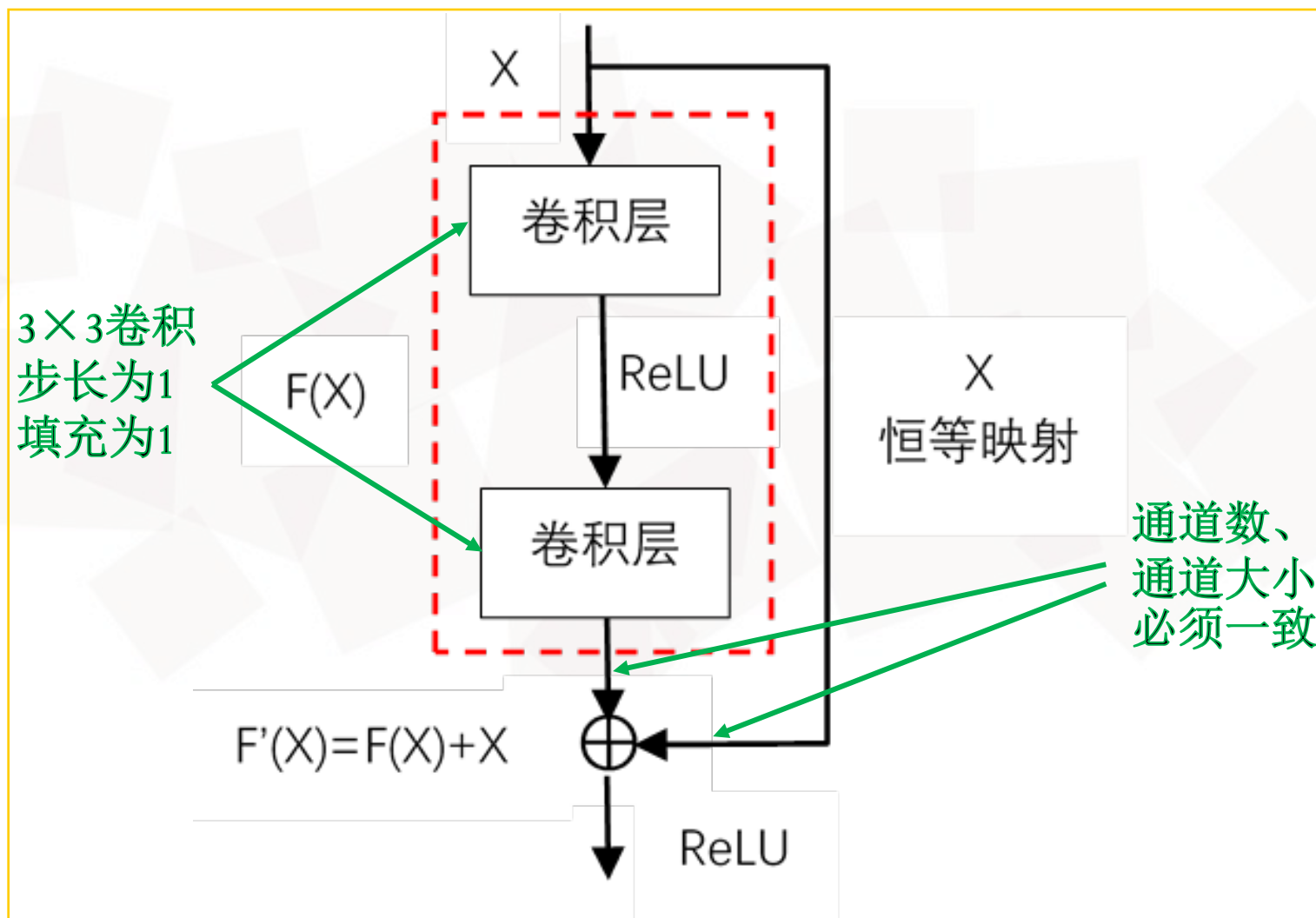




解决思路



残差模块



残差网络 (ResNet)

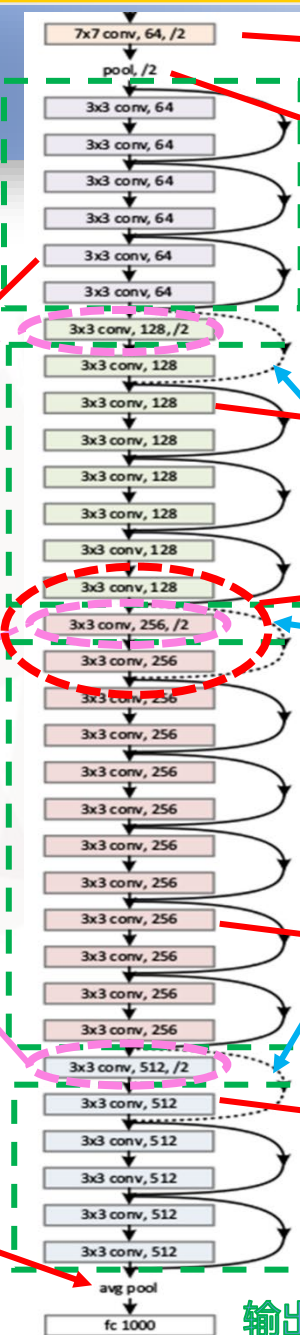
3个残差模块

**64个 3×3 卷积核，
步长为1，填充为1。**

步长为2，长宽各减一半，通道数增加了一倍。恒等映射需要等维处理。

全局平均池化

输入:
 $224 \times 224 \times 3$



64个 7×7 卷积核，
步长为2，填充为3。

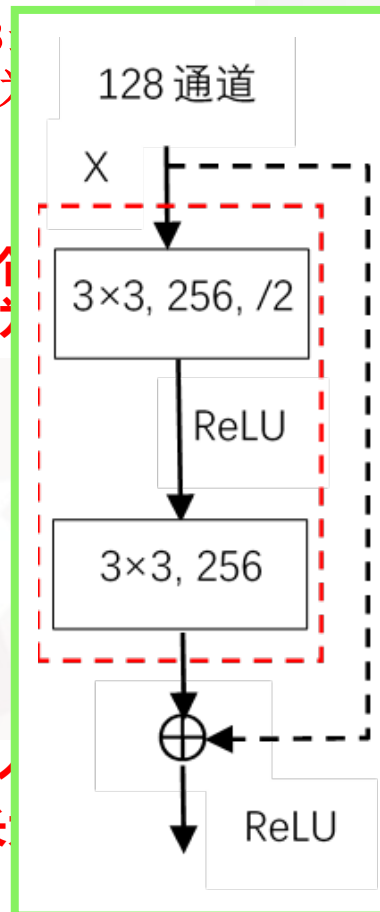
步长为

128个
步长为

虚线

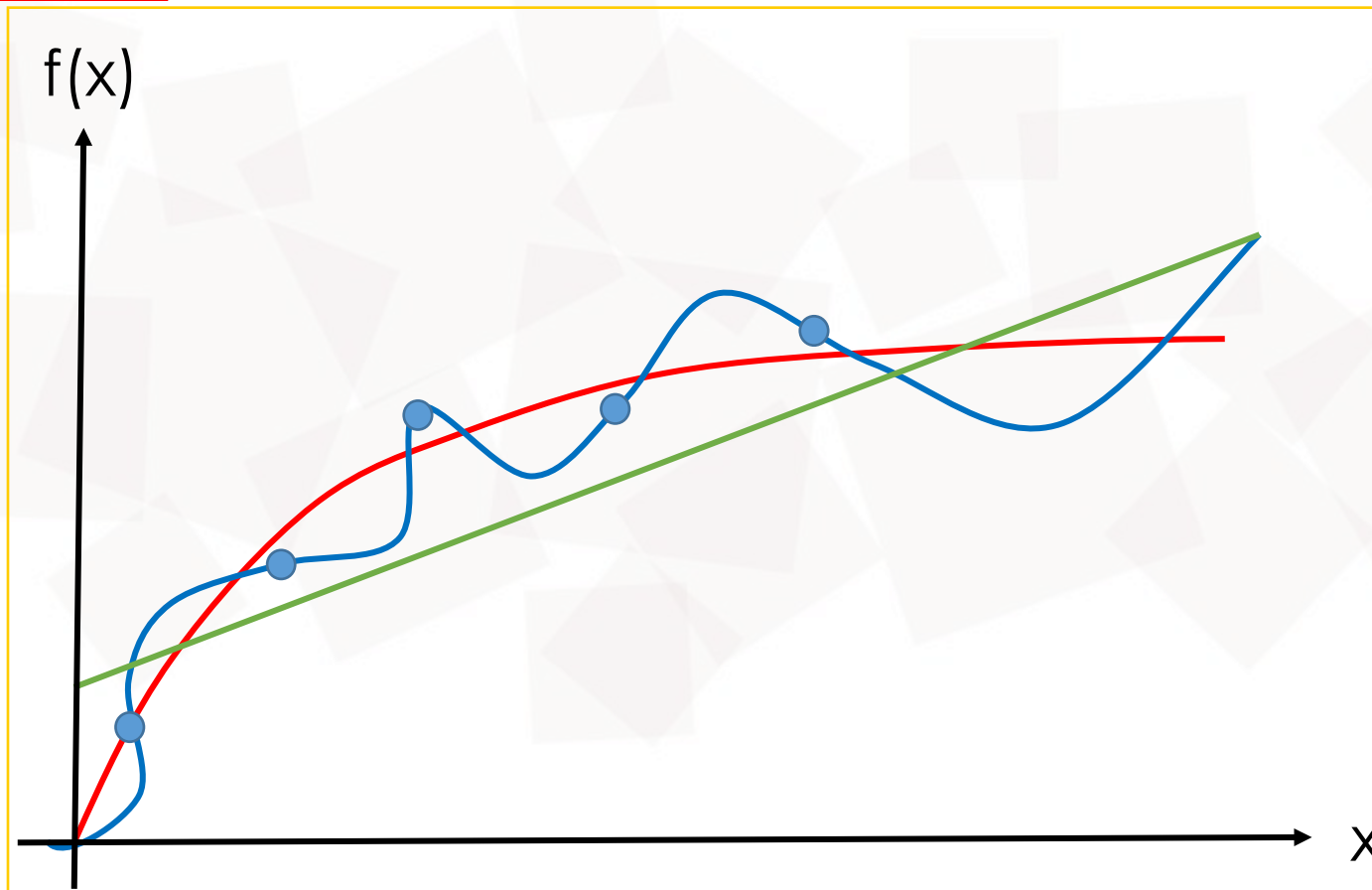
256
步长

512个 3×3 卷积核，步长为1，填充为1。

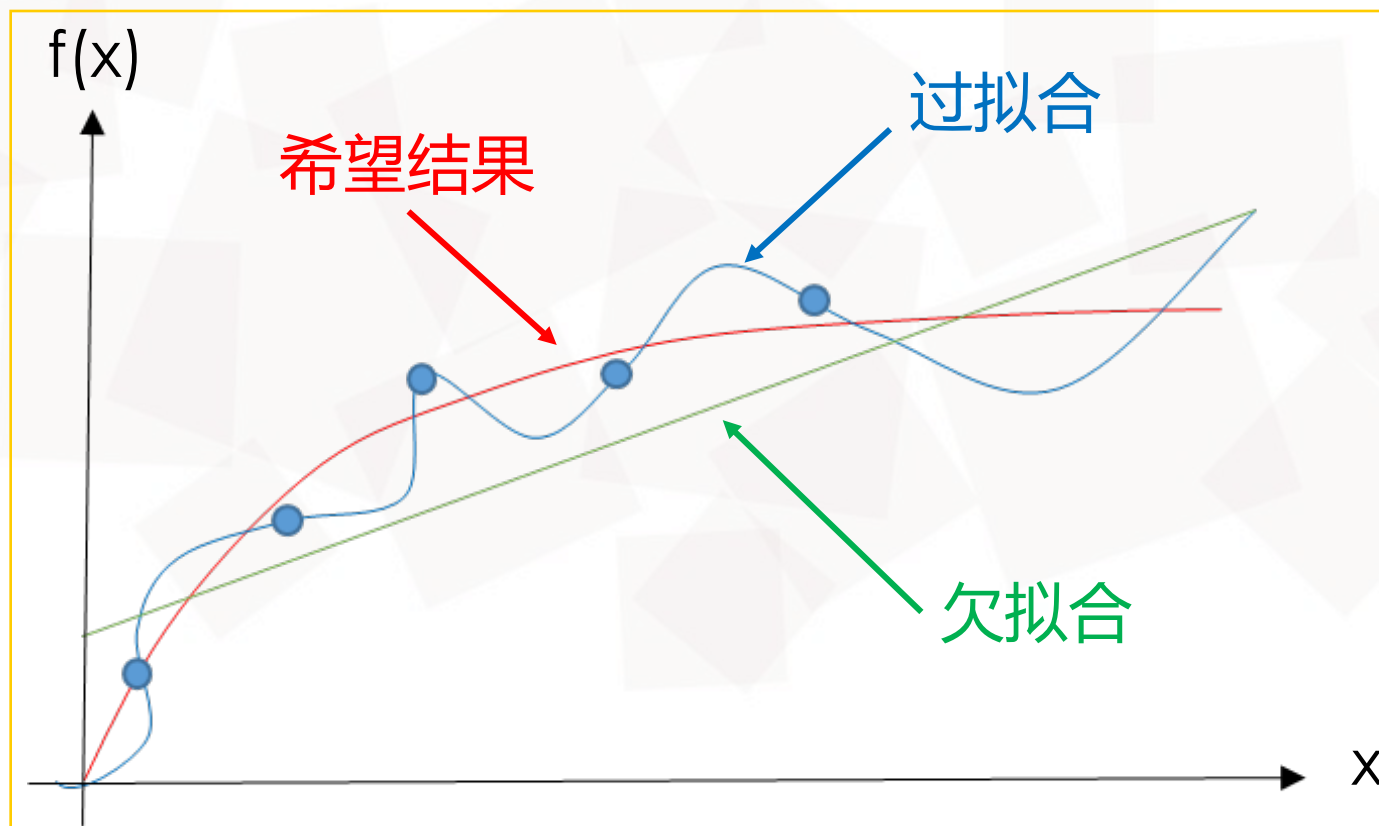


输出:
1000

什么是过拟合问题

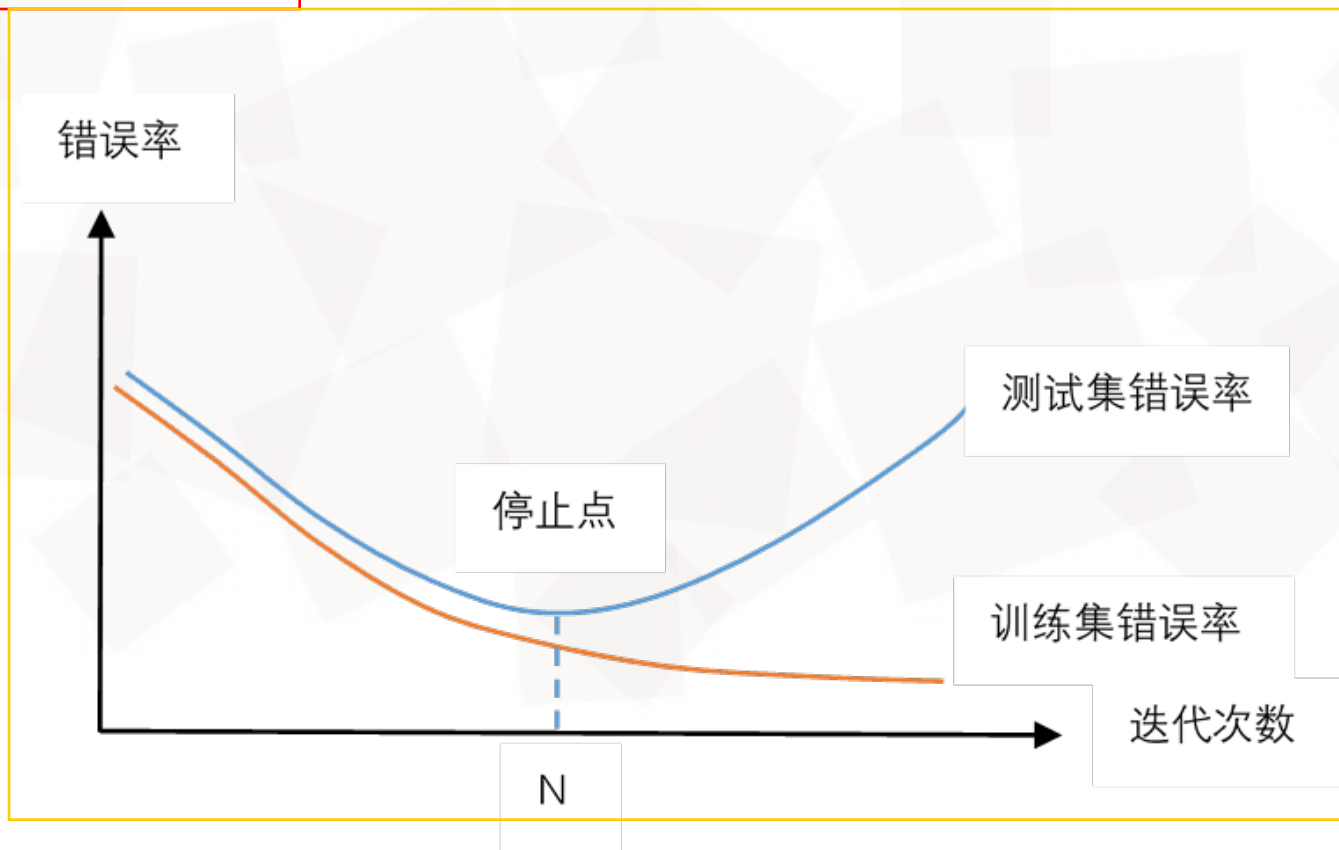


什么是过拟合问题





神经网络的过拟合问题





减少过拟合的方法：正则化项法

- 误差平方和损失函数

$$E_d(w) = \sum_{k=1}^M (t_{kd} - o_{kd})^2$$

- 加正则化项

$$E_d(w) = \sum_{k=1}^M (t_{kd} - o_{kd})^2 + \|w\|_2^2$$

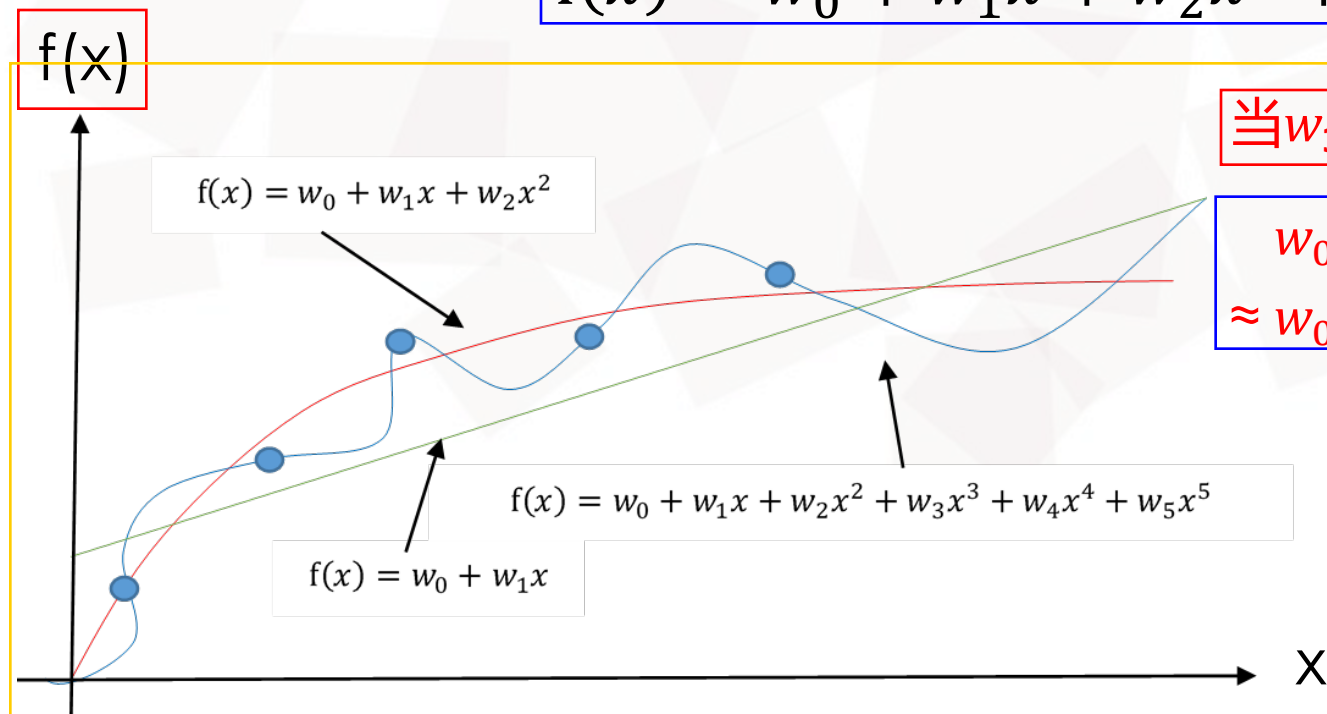
$$\|w\|_2^2 = w_1^2 + w_2^2 + \cdots + w_N^2$$

- 说明：并不限于误差平方和损失函数和2-范数

正则化项的作用：降低模型复杂性

- 曲线的多项式表示：

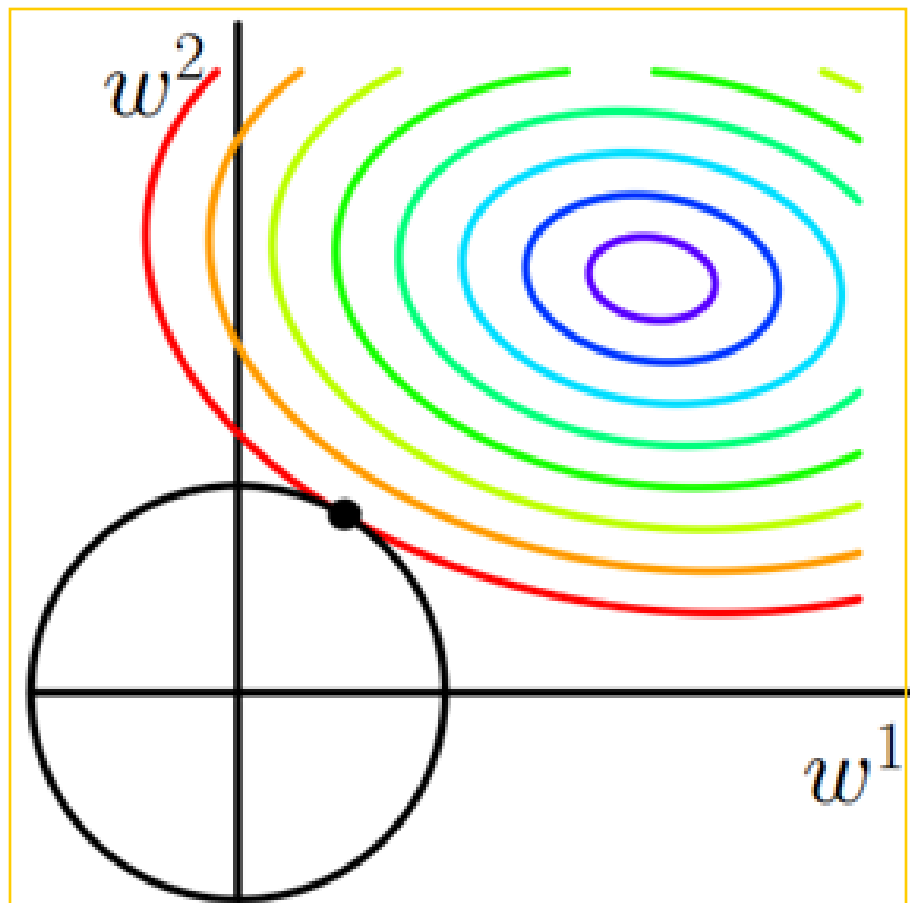
$$f(x) = w_0 + w_1x + w_2x^2 + \cdots w_nx^n$$



当 w_3 、 w_4 、 w_5 比较小时：

$$w_0 + w_1x + w_2x^2 + w_3x^3 + w_4x^4 + w_5x^5 \approx w_0 + w_1x + w_2x^2$$

L2 (2-范数) 正则化项



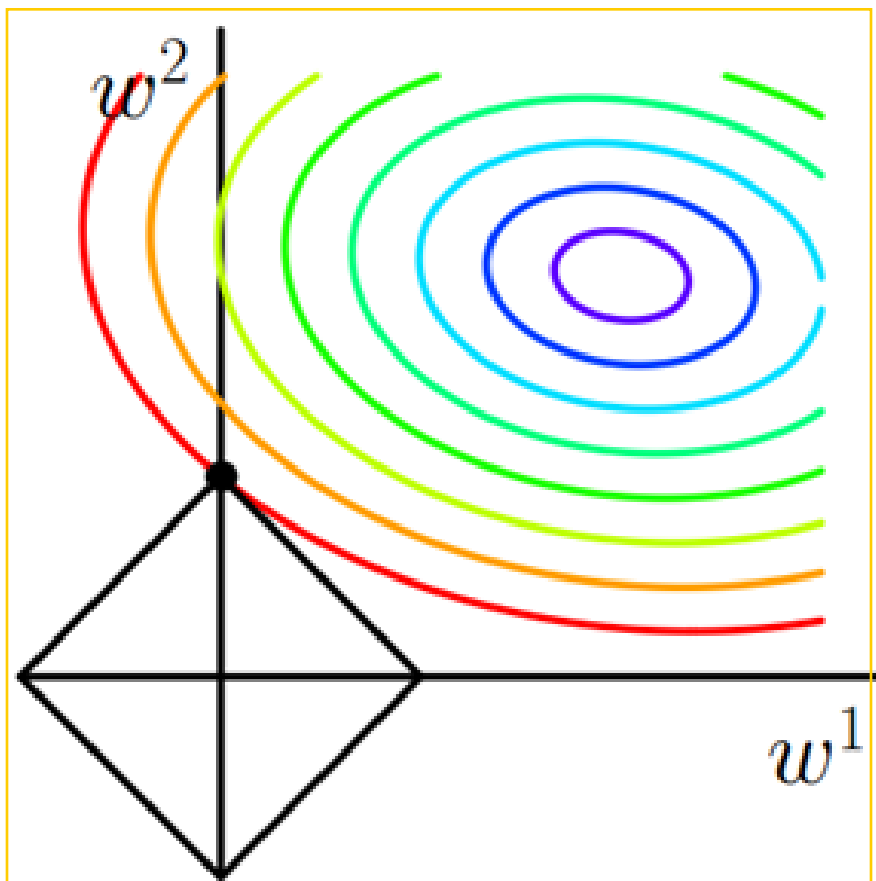
2-范数:

$$\|w\|_2^2 = \sum_i w_i^2$$

效果:

很多参数值很小, 但基本
不会为0。抗干扰能力强。

L1 (1-范数) 正则化项



1-范数:

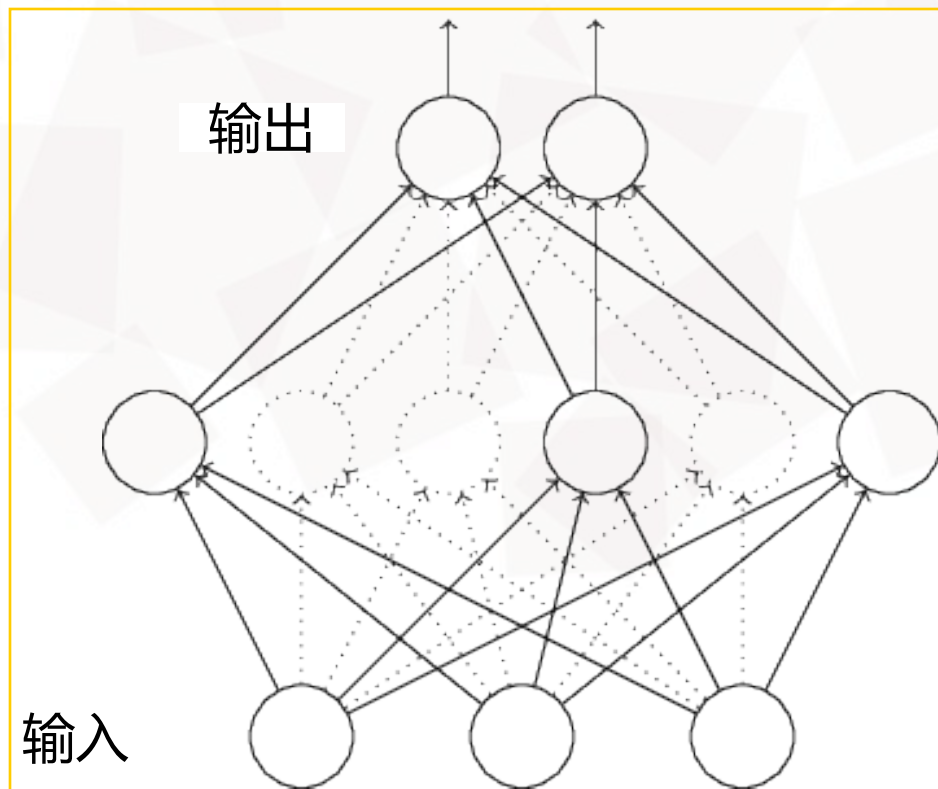
$$\|w\|_1 = \sum_i |w_i|$$

效果:

一些参数为0, 起到特征选择的作用。

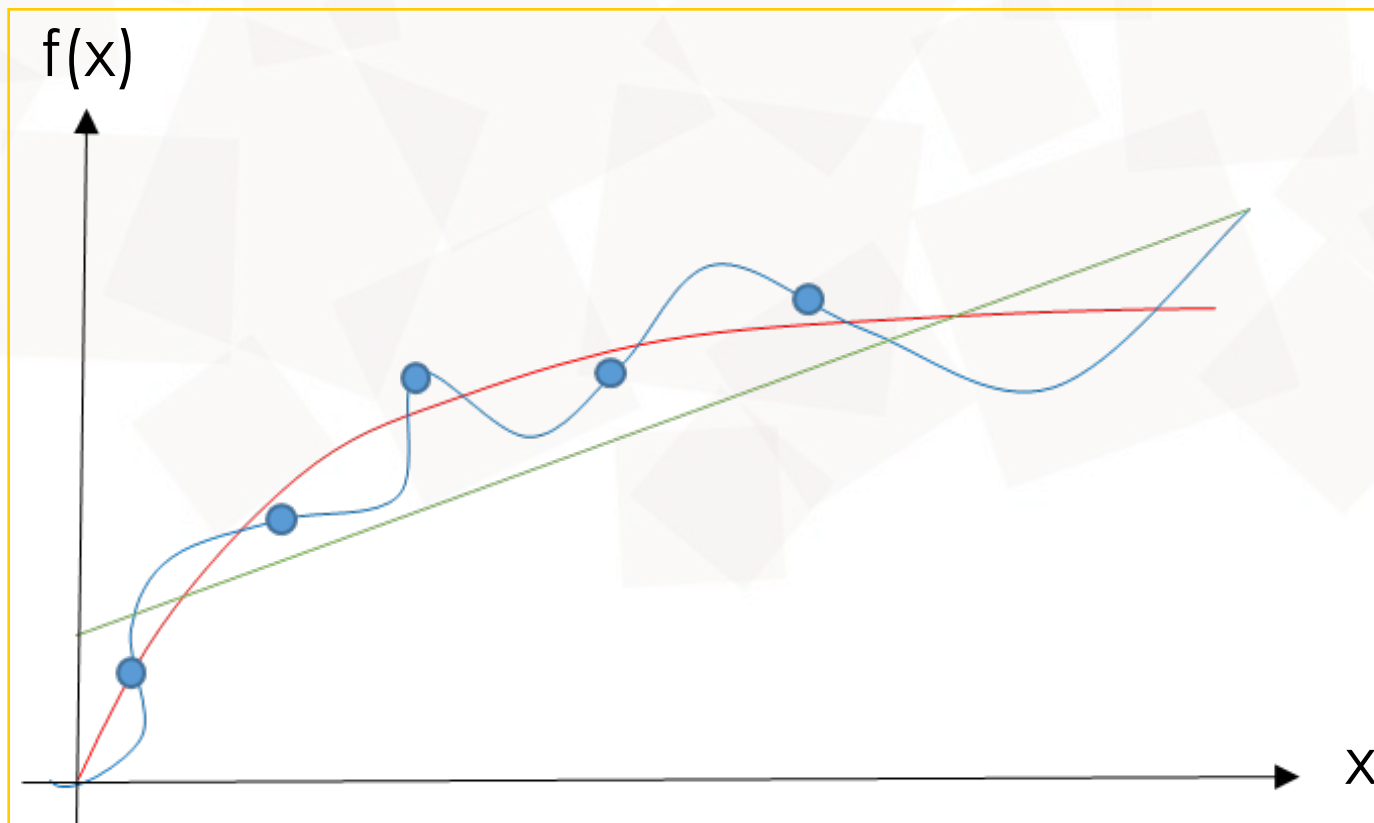
减少过拟合的方法：舍弃法（Dropout）

- 随机地临时舍弃一些神经元，每次训练一个神经元子集



减少过拟合的方法：数据增强法

- 数据越多，过拟合的风险就越小





如何获得更多的数据？

- 数据增强

- 通过各种变换增加数据的数量

- 图像数据

- 缩放、旋转、局部截取、改变颜色等

如何获得更多的数据?





本课件源自清华大学马少平教授编著的《深入浅出人工智能》配套课件。